ENGINEERING PLAYBOOKS · PRODUCTION SERIES

FIRST EDITION · 2.0

The Complete Software Engineering Handbook

Build Any Production-Grade Application End-to-End — Complete Code, Diagrams, and Checklists for Freshers to Senior Engineers

- ✓ 10-day zero → production build path
- ✓ 17 parts — every section has diagram → complete code → checklist
- ✓ 100% runnable, copy-paste TypeScript code (not code shapes)
- ✓ Full stack: Next.js · Fastify · Prisma · PostgreSQL · Redis · Docker · AWS
- ✓ 25 troubleshooting fixes + Error Decoder appendix

For freshers, junior engineers, and anyone shipping real production software

The Complete Software Engineering Handbook

ENGINEERING PLAYBOOKS

The Complete Software Engineering Handbook

Build Any Production-Grade Application End-to-End — Complete Code, Diagrams, and Checklists for Freshers to Senior Engineers

Complete Edition

Version 2.0

Engineering Playbooks

2026

Copyright

Copyright © 2026 Engineering Playbooks. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or without prior written permission, except for brief quotations in reviews or internal team training with attribution.

ISBN: 978-0-000000-00-0 (placeholder — assign before retail print)

Edition: Complete Edition (2.0)

Publisher: Engineering Playbooks

Printed / distributed: Digital PDF — 2026

Disclaimer

This handbook is for educational and engineering guidance. Stack choices, cloud pricing, and security threats change over time. Always verify against your organization's policies, compliance requirements, and official vendor documentation before production deployments.

Trademarks

All product names, frameworks, and tools mentioned are trademarks of their respective owners. Next.js, React, PostgreSQL, Docker, Kubernetes, GitHub Actions, and others belong to their respective companies.

Printed in digital form · 2026

About This Book

The Complete Software Engineering Handbook is the definitive action-first guide for building production-grade applications end-to-end — the same way senior engineers ship at real companies. Every section follows the same pattern: *architecture diagram → complete runnable code → checklist*.

Unlike theory-heavy textbooks or guides that show you the "shape" of the answer, every code block in this book is a complete, copy-paste-ready file with correct imports and TypeScript types. Zero gap-filling required.

Who this book is for

- Complete beginners with zero backend experience
- Fresher and junior developers preparing for industry
- Self-taught builders who want production discipline
- Teams needing one shared engineering playbook

What you will be able to do after this book

- Follow a 10-day plan from zero to a deployed production app
- Build a full stack with Next.js, Fastify, Prisma, and PostgreSQL
- Implement JWT auth with refresh tokens, bcrypt, and session management
- Write unit, integration, and E2E tests with Vitest and Playwright
- Containerize with multi-stage Docker, CI/CD with GitHub Actions
- Deploy to Railway/Fly.io or AWS ECS with Terraform
- Add observability with Pino logs and OpenTelemetry traces
- Troubleshoot the 25 most common production failures

Technology stack

Next.js 15 · TypeScript · Fastify · Prisma · PostgreSQL · Redis · Docker · GitHub Actions · AWS

How to Use This Handbook

Tip: In this PDF, blue links in the Table of Contents and Index jump to the section.

If you want to...	Start here
Build your first app in 10 days	PART 0
Understand the full system	Architecture diagram
Fix a broken deploy or API	PART 13
Ship safely to users	PART 12
Copy terminal commands	Appendix A

Reading modes

1. **Fast path (recommended):** Follow PART 0 day-by-day only. Skim the rest when stuck.
2. **Reference mode:** Jump to the PART that matches your task (frontend, DB, DevOps).
3. **Visual mode:** Study diagrams first, then read tables and checklists.

Table of Contents

Click any title below to jump to that section.

FRONT MATTER

Front Cover 1

Half Title 2

Title Page 2

Copyright 3

About This Book 4

How to Use This Handbook 5

Table of Contents 6

List of Figures 7

List of Tables 8

BODY

How to read this (60 seconds) 10

The whole system in one picture 11

PART 0 — 10-Day Build Plan (Zero → Production) 11

Day 1 — Scaffold ("it runs") 12

PART 1 — Local Development Setup 15

1.1 What you need installed 15

PART 2 — Architecture & Stack Selection 18

2.1 Pick your app shape first 19

PART 3 — Frontend Engineering 23

3.1 Next.js rendering — which mode? 23

PART 4 — Backend Engineering 28

4.1 Backend architecture — the layers 28

PART 5 — Database Engineering 34

5.1 Prisma schema — define your data 34

PART 6 — Auth & Security 38

6.1 JWT auth flow 38

PART 7 — Testing 42

7.1 The testing pyramid 42

PART 8 — DevOps & CI/CD 45

8.1 Why Docker? 45

PART 9 — Cloud & Deploy 49

9.1 Easiest path — Railway or Fly.io (start here) 49

PART 10 — Observability & Monitoring 51

10.1 The three pillars 51

PART 11 — Performance 52

11.1 Find slowness before optimizing 53

PART 12 — Production Launch Checklist 54

Critical (must be green before shipping) 55

PART 13 — Troubleshooting (25 fixes) 56

PART 14 — Real-World App Blueprints 58

Blueprint A — SaaS MVP (the standard fresher project) 58

PART 15 — Team & Git Workflow 60

Branch and PR rules 60

PART 16 — Think Like a Senior Engineer 61

The 5 rules 61

PART 17 — 90-Day Growth Path 62

Appendix A — Copy-Paste Commands 62

Appendix B — Environment Variables Reference 63

Appendix C — Error Decoder 64

Appendix D — Glossary 65

You are ready 66

BACK MATTER

Index 67

Colophon 68

> **Start here:** About This Book → PART 0 (10-day plan). All blue links are clickable in this PDF.

List of Figures

Click a figure title or page number to jump to that diagram.

No.	Description	Page
Figure 1	Architecture & flow diagram 1	10
Figure 2	Architecture & flow diagram 2	11
Figure 3	Architecture & flow diagram 3	12
Figure 4	Day 2 — CRUD (one feature end-to-end)	12
Figure 5	Day 4 — Auth (register / login / JWT)	13
Figure 6	Day 5 — Tests	14
Figure 7	Day 6 — Docker (multi-stage)	14
Figure 8	Day 7 — GitHub Actions CI	15
Figure 9	2.1 Pick your app shape first	19
Figure 10	2.2 Backend language decision tree	19
Figure 11	2.3 Frontend framework decision tree	20
Figure 12	2.4 Database decision tree	21
Figure 13	2.5 The 2026 production stack (opinionated defaults)	22
Figure 14	3.1 Next.js rendering — which mode?	23
Figure 15	3.3 State management — the full decision guide	24
Figure 16	4.1 Backend architecture — the layers	28
Figure 17	5.3 Indexing — speed up your queries	36
Figure 18	5.5 Migrations — never break production	36
Figure 19	6.1 JWT auth flow	38
Figure 20	7.1 The testing pyramid	42
Figure 21	8.1 Why Docker?	45
Figure 22	8.4 Deployment strategies	48
Figure 23	9.2 AWS — production-grade setup	49
Figure 24	10.1 The three pillars	51
Figure 25	11.1 Find slowness before optimizing	53

No.	Description	Page
Figure 26	11.3 Caching layers	53
Figure 27	11.5 Performance checklist	55
Figure 28	Launch readiness score	56
Figure 29	Blueprint A — SaaS MVP (the standard fresher project)	58
Figure 30	Blueprint B — E-commerce (simplified)	58
Figure 31	System design thinking (for interviews + real work)	59
Figure 32	System design thinking (for interviews + real work)	60
Figure 33	The 5 rules	61
Figure 34	Architecture Decision Records (ADR)	62

List of Tables

Click a location to jump to that section.

Topic	Location
10-day build plan (day-by-day)	PART 0
Stack decision guide	PART 2
Node vs Python vs Go vs Java	PART 2
Frontend framework choice	PART 2
Best tools, packages & libraries (2026)	PART 2
HTTP methods cheat sheet	PART 4
SQL vs NoSQL decision	PART 5
Scaling ladder	PART 11
25 troubleshooting fixes	PART 13
Production launch checklist	PART 12
90-day growth path	PART 17
Environment variables reference	See topic
Error decoder (13 common errors)	See topic
Glossary	See topic

The Complete Software Engineering Handbook

> **Zero to Production — Everything a Fresher Needs to Build Any Real App**

> Every section gives you: **the diagram first, then complete runnable code, then the checklist.**
> No gaps. No "fill this in yourself." Copy → paste → ship.

I want to...	Go to...
Start today (day 0)	PART 0 — 10-day plan
Set up my machine	PART 1 — Local setup
Pick a stack	PART 2 — Architecture & stack
Build the frontend	PART 3 — Frontend
Build the backend	PART 4 — Backend
Design the database	PART 5 — Database
Add auth & security	PART 6 — Auth & security
Write tests	PART 7 — Testing
Containerize + CI	PART 8 — DevOps & CI/CD
Deploy to cloud	PART 9 — Cloud & deploy
Add monitoring	PART 10 — Observability
Tune performance	PART 11 — Performance
Launch safely	PART 12 — Launch checklist
Fix something broken	PART 13 — Troubleshooting
See real app patterns	PART 14 — Blueprints
Work with a team	PART 15 — Team & Git
Think like a senior	PART 16 — Senior mindset
Plan your growth	PART 17 — 90-day path
Copy-paste commands	Appendix A
Look up env vars	Appendix B
Decode an error	Appendix C — Error decoder
Learn a term	Appendix D — Glossary

How to read this (60 seconds)

Rules that save hours:

1. **Do before reading** — follow PART 0 day-by-day; use other parts as reference

2. **Diagrams + bold tables = must-read** — long paragraphs = skim or skip
3. **One working slice at a time** — health check → CRUD → auth → tests → deploy
4. **If stuck** — go to PART 13 (Troubleshooting) or Appendix C (Error decoder)

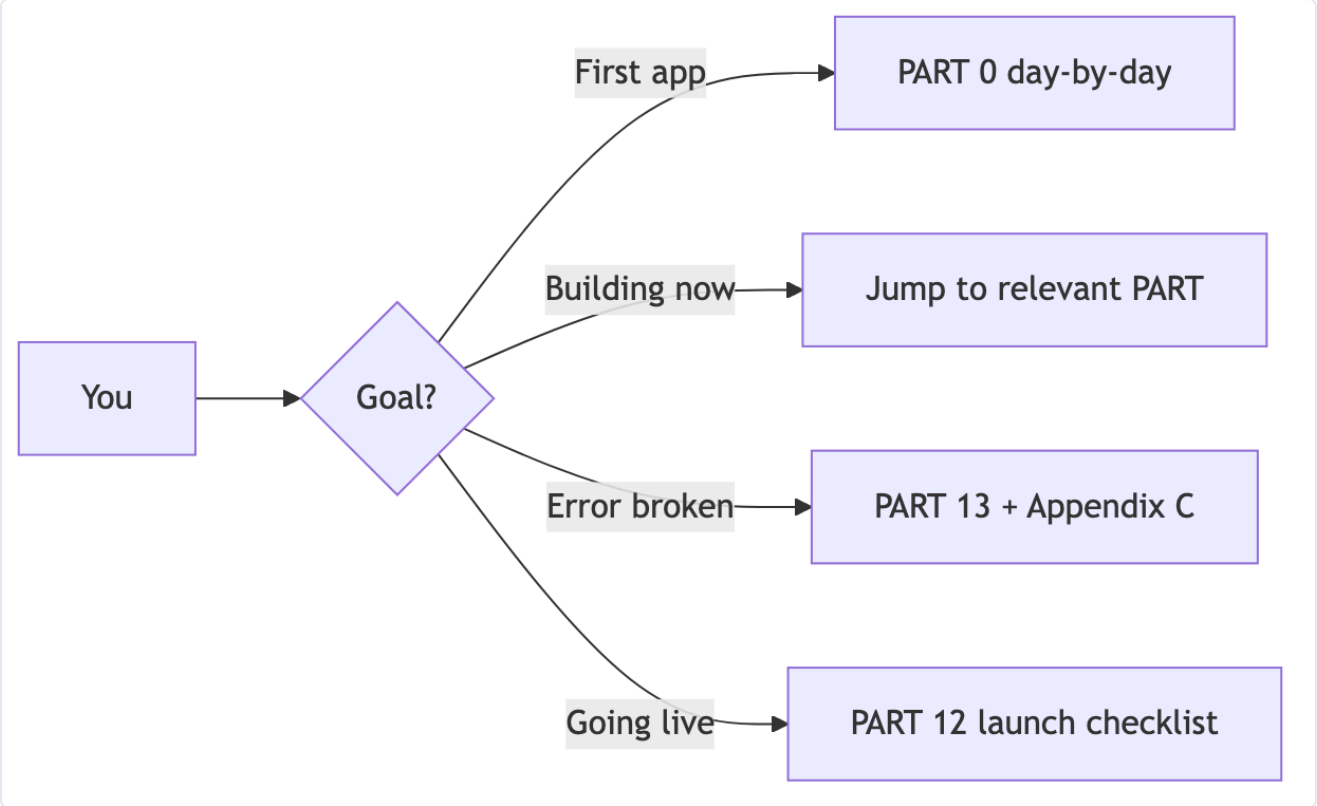


Figure 1 — Architecture & flow diagram 1

The whole system in one picture

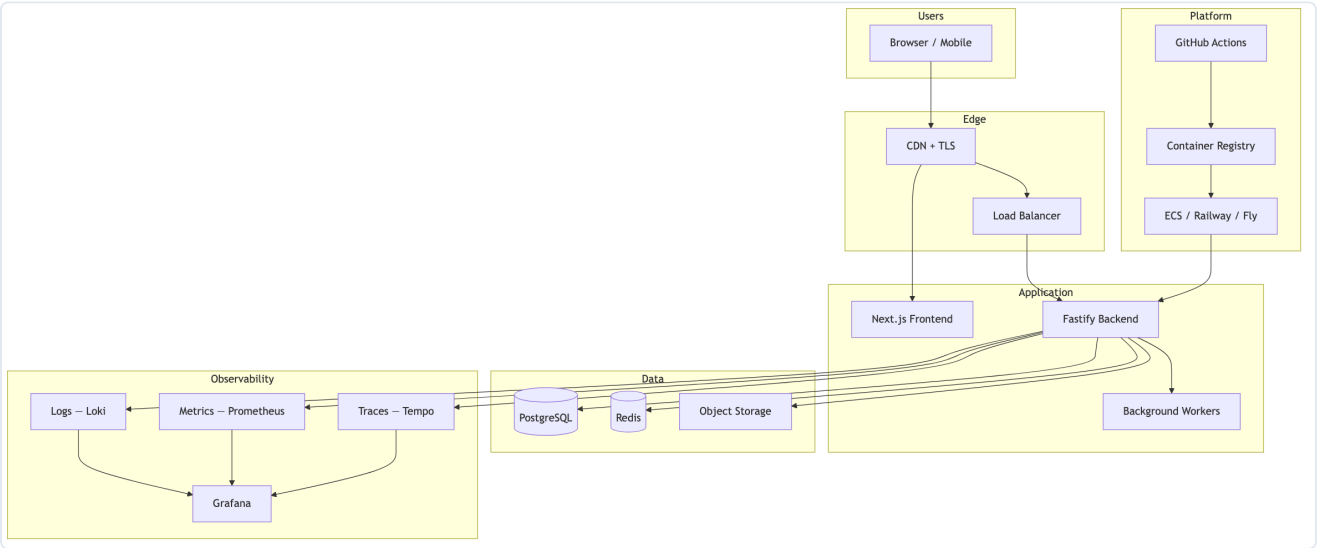


Figure 2 — Architecture & flow diagram 2

> Save this diagram. When confused about where something lives, come back here.

PART 0 — 10-Day Build Plan (Zero → Production)

> **Outcome after 10 days:** A deployed app with CRUD, auth, tests, Docker, CI, health checks, and basic monitoring.

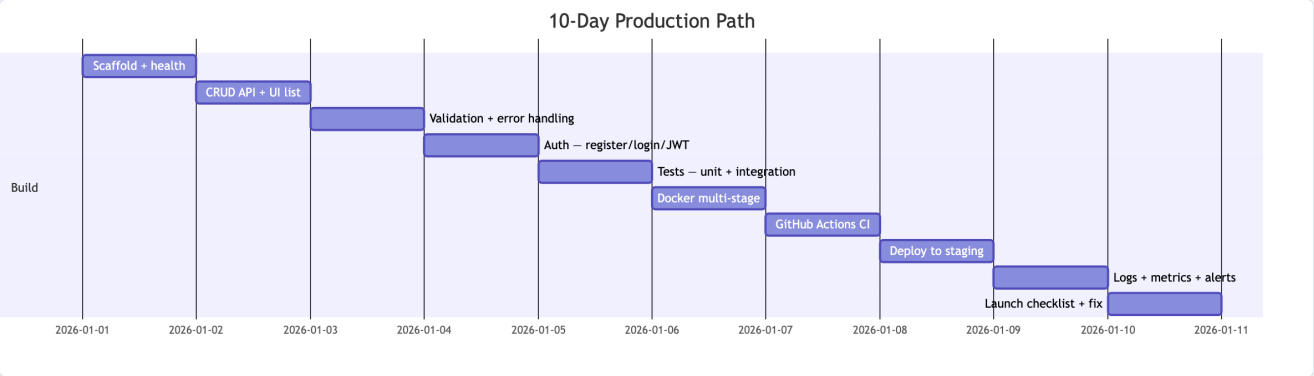


Figure 3 — Architecture & flow diagram 3

Day 1 — Scaffold ("it runs")

Step	Action	Done
1	Clone / copy `starter-app/`	☐
2	`cp .env.example .env`	☐
3	`docker-compose up --build` — all green	☐
4	`curl http://localhost:4000/health` → `{"status":"ok"}`	☐
5	Open `http://localhost:3000` in browser	☐
6	First commit: `chore: initial scaffold`	☐

```
cd starter-app
cp .env.example .env
docker-compose up --build

# In another terminal:
curl -s http://localhost:4000/health
# Should print: {"status":"ok"}
```

Day 2 — CRUD (one feature end-to-end)

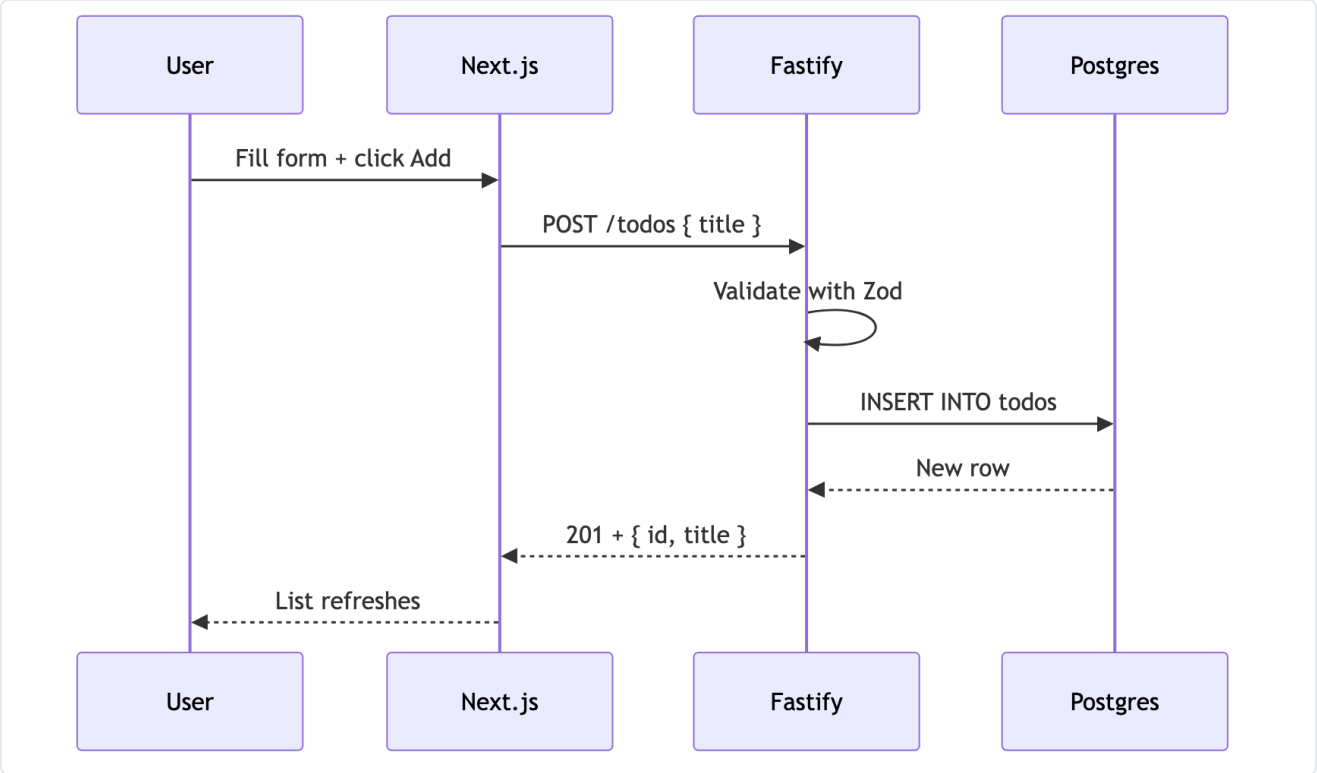


Figure 4 — Day 2 — CRUD (one feature end-to-end)

Step	Action	Done
1	Add `todos(id, title, completed, created_at)` to DB schema	☐
2	API: `GET /todos`, `POST /todos`, `PATCH /todos/:id`	☐
3	Frontend: list + form + toggle button	☐
4	Test with curl: create → list → toggle	☐
5	Commit: `feat: add todos CRUD`	☐

Day 3 — Validation + consistent errors

Rule	What to do
Never trust client input	Validate every write endpoint with Zod
Same error shape	`{ "error": "message", "code": "VALIDATION_ERROR" }`
Right HTTP code	`400` bad input · `401` no auth · `404` not found · `500` crash

```
// Every write route follows this pattern:
const body = createTodoSchema.parse(request.body)
// If parse fails, Zod throws → your error handler catches → 400
```

Day 4 — Auth (register / login / JWT)

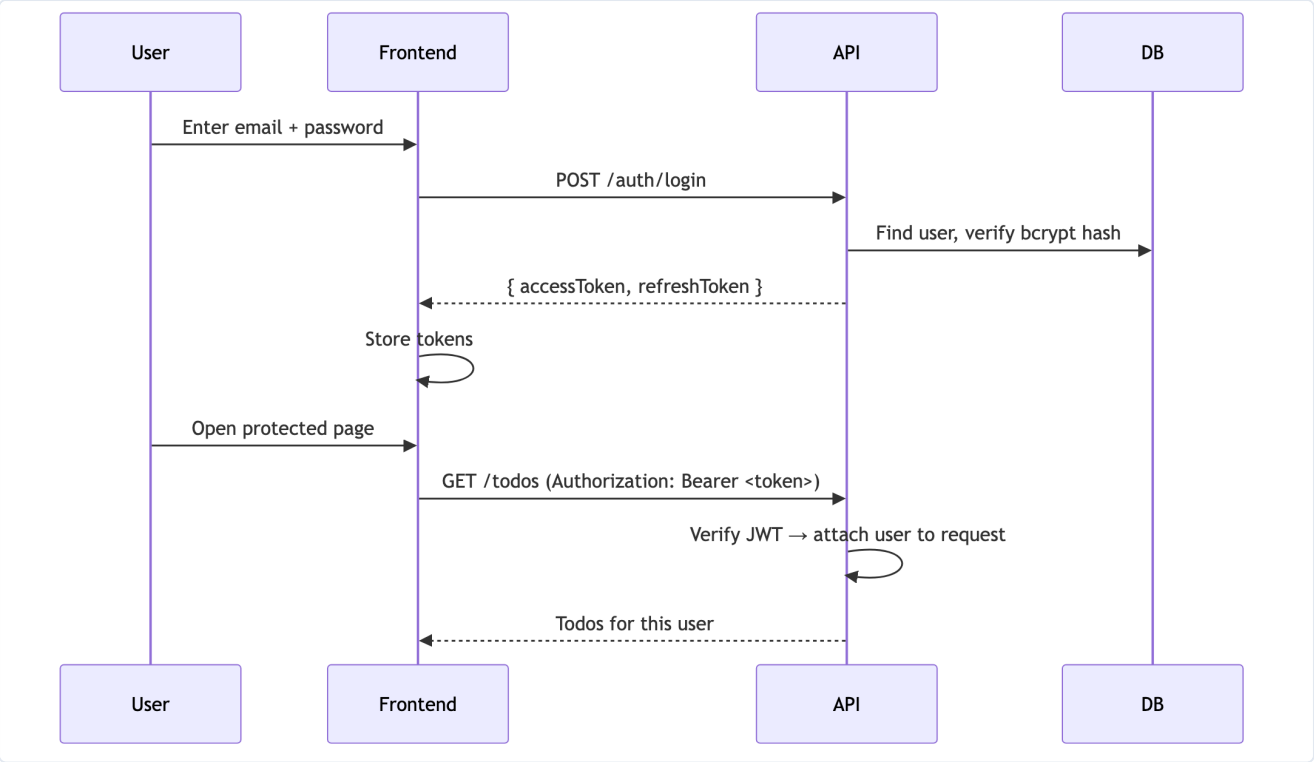


Figure 5 — Day 4 — Auth (register / login / JWT)

Step	Action	Done
1	`POST /auth/register` — hash password, return tokens	☐
2	`POST /auth/login` — verify hash, return tokens	☐
3	`GET /auth/me` — protected route, return user profile	☐
4	Auth middleware — reject missing/invalid JWT with 401	☐
5	Protect `/todos` routes — only your own todos	☐

Day 5 — Tests

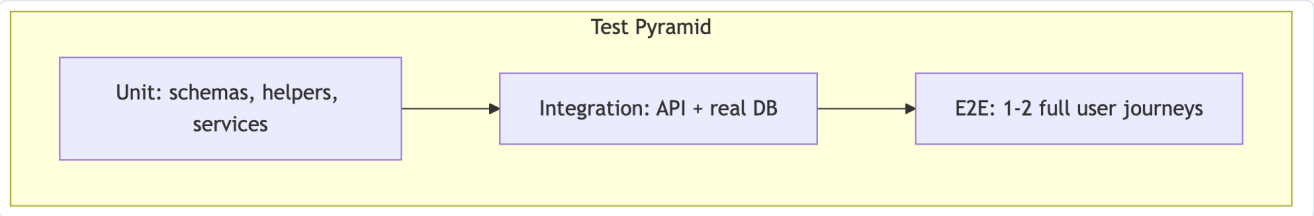


Figure 6 — Day 5 — Tests

Layer	Test	File
Unit	Zod schema rejects empty title	`src/tests/validation.test.ts`
Integration	`POST /todos` → 201 with auth token	`src/tests/todos.test.ts`
E2E	Login → create todo → see in list	`e2e/todos.spec.ts`

Day 6 — Docker (multi-stage)



Figure 7 — Day 6 — Docker (multi-stage)

Check	☐
`.dockerignore` excludes `node_modules` and `.env`	
Multi-stage build — final image has no build tools	
Non-root user: `USER node`	
`HEALTHCHECK` hits `/health`	

Day 7 — GitHub Actions CI



Figure 8 — Day 7 — GitHub Actions CI

Goal: if `npm test` fails, the CI fails. Nobody merges broken code.

Day 8 — Deploy to staging

Step	Action
1	Set env secrets in hosting platform (never in git)
2	Run DB migrations BEFORE deploying new app version
3	Smoke test: `curl https://staging.yourapp.com/health`
4	Document rollback: what image tag to redeploy

Day 9 — Observability basics

Signal	Minimum for Day 9
Logs	JSON with `requestId`, `level`, `msg`
Health	`/health` checks DB connectivity
Alert	Slack/email on 5xx spike or health failure

Day 10 — Launch checklist

Run PART 12. If all critical boxes are green → **ship**.

PART 1 — Local Development Setup

Goal: running local environment in under 15 minutes.

1.1 What you need installed

```
# Check versions
node --version    # Need 20+
docker --version  # Need 24+
git --version     # Need 2.40+

# Install on Mac (Homebrew)
brew install node@22
brew install --cask docker
brew install git

# Install global CLI tools
npm install -g pnpm   # faster than npm (optional but recommended)
```

1.2 Project folder structure

```
my-app/
├── frontend/                ← Next.js app
│   ├── src/
│   │   ├── app/             ← Next.js App Router
│   │   ├── components/      ← Reusable UI components
│   │   ├── hooks/           ← Custom React hooks
│   │   ├── lib/             ← API client, utilities
│   │   └── types/           ← TypeScript types
│   ├── .env.local
│   └── package.json
│
├── api/                     ← Fastify backend
│   ├── src/
│   │   ├── routes/          ← Route handlers (thin)
│   │   ├── services/        ← Business logic
│   │   ├── repositories/    ← Database queries
│   │   ├── middleware/      ← Auth, rate limit, logging
│   │   ├── plugins/         ← Fastify plugins
│   │   ├── schemas/         ← Zod validation schemas
│   │   └── index.ts         ← App entry point
│   ├── prisma/
│   │   ├── schema.prisma    ← DB schema
│   │   └── migrations/      ← Migration files
│   ├── .env
│   └── package.json
│
├── docker-compose.yml       ← Local DB + all services
├── .github/
│   └── workflows/ci.yml     ← CI pipeline
└── README.md
```

1.3 docker-compose.yml — start all local services

```
# docker-compose.yml
version: '3.9'

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: myapp
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U myapp"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data

  api:
    build: ./api
    ports:
      - "4000:4000"
    environment:
      DATABASE_URL: postgresql://myapp:password@postgres:5432/myapp
      REDIS_URL: redis://redis:6379
      JWT_SECRET: dev-secret-change-in-production
      NODE_ENV: development
    depends_on:
      postgres:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:4000
    depends_on:
      - api

volumes:
  postgres_data:
  redis_data:
```

```
# Start everything
docker-compose up --build

# Stop everything
docker-compose down

# Wipe data and restart fresh
docker-compose down -v && docker-compose up --build
```

1.4 Environment variables — never get this wrong

```
# api/.env
DATABASE_URL="postgresql://myapp:password@localhost:5432/myapp"
REDIS_URL="redis://localhost:6379"
JWT_SECRET="change-this-in-prod-use-crypto-random"
JWT_REFRESH_SECRET="also-change-this-in-prod"
PORT=4000
NODE_ENV=development

# Generate a secure secret:
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

```
# frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

> **RULE:** Never commit `.env` files. Always commit `.env.example` with empty values.

```
# Add to .gitignore
echo ".env" >> .gitignore
echo ".env.local" >> .gitignore
echo ".env.*.local" >> .gitignore
```

1.5 Scaffold the frontend

```
npx create-next-app@latest frontend \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir \
  --import-alias "@/*"

cd frontend

# Essential packages
npm install @tanstack/react-query zustand
npm install react-hook-form zod @hookform/resolvers
npm install axios

# Component library (saves weeks of UI work)
npx shadcn-ui@latest init
npx shadcn-ui@latest add button input form card table badge

npm run dev
# Open http://localhost:3000
```

1.6 Scaffold the backend

```
mkdir api && cd api
npm init -y

# Core packages
npm install fastify @fastify/cors @fastify/helmet
npm install jsonwebtoken   # use jsonwebtoken directly — NOT @fastify/jwt
npm install @prisma/client
npm install zod
npm install bcrypt jsonwebtoken ioredis
npm install pino pino-pretty

# Dev dependencies
npm install -D typescript tsx ts-node @types/node prisma
npm install -D @types/bcrypt @types/jsonwebtoken
npm install -D vitest @vitest/coverage-v8

# Initialize Prisma (creates prisma/schema.prisma)
npx prisma init
```

1.7 TypeScript config (copy this — don't use the default)

```
// api/tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

1.8 package.json scripts (copy this)

```
// api/package.json (scripts section)
{
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:coverage": "vitest run --coverage",
    "db:migrate": "prisma migrate dev",
    "db:deploy": "prisma migrate deploy",
    "db:studio": "prisma studio",
    "db:generate": "prisma generate"
  }
}
```

```
// frontend/package.json (scripts section)
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "type-check": "tsc --noEmit"
  }
}
```

> **Why `tsx watch` for dev?** It runs TypeScript directly without a compile step — changes reflect instantly. In production you compile to `dist/` with `tsc` and run `node dist/index.js`.

```
npm run dev
# API runs at http://localhost:4000
```

PART 2 — Architecture & Stack Selection

2.1 Pick your app shape first

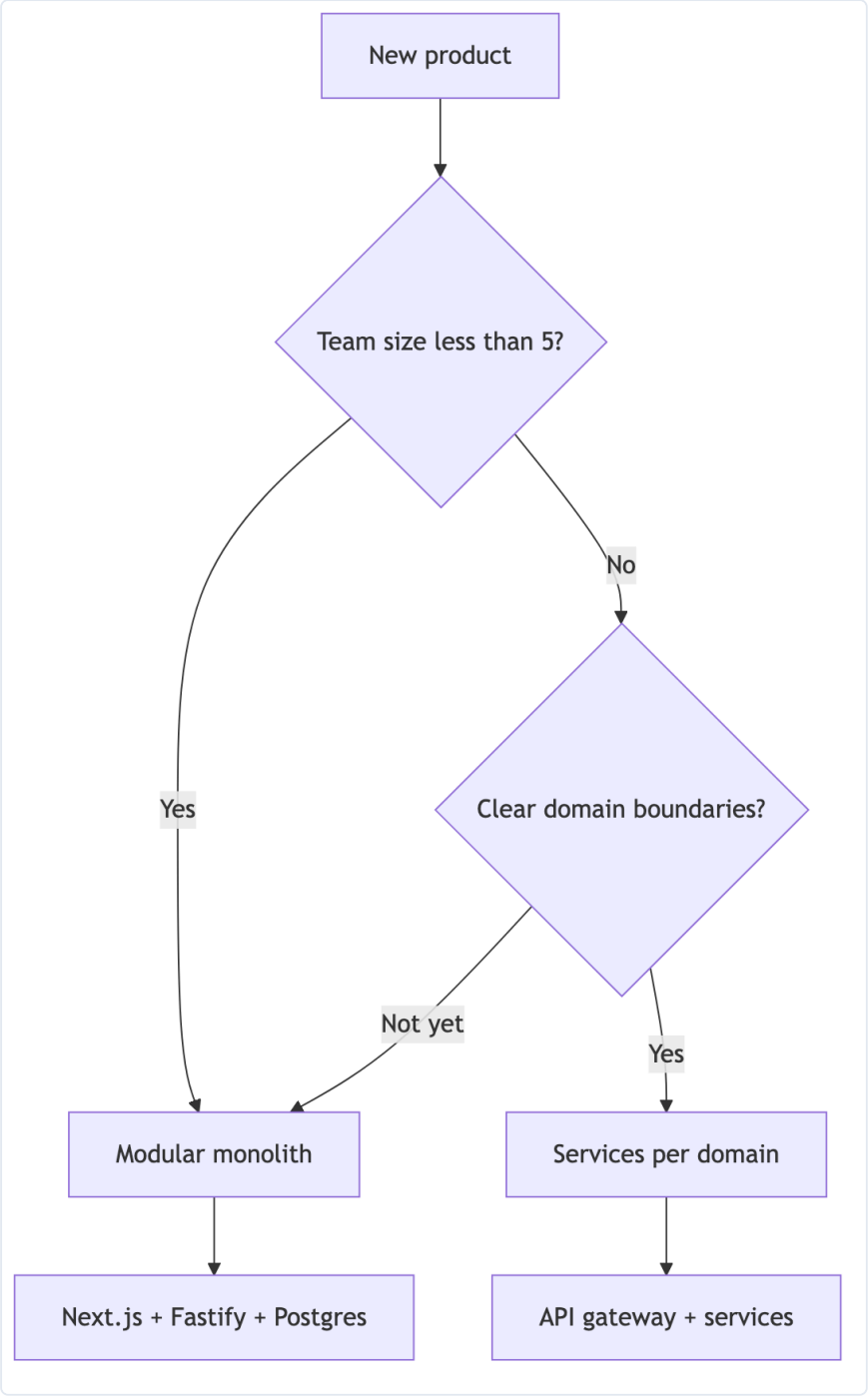


Figure 9 — 2.1 Pick your app shape first

Pattern	When	Fresher default
Modular monolith	MVP, small team	✅ Start here always
Microservices	Large team, proven domains	❌ Way later
Serverless	Spiky traffic, ops-light	Optional for side projects

2.2 Backend language decision tree

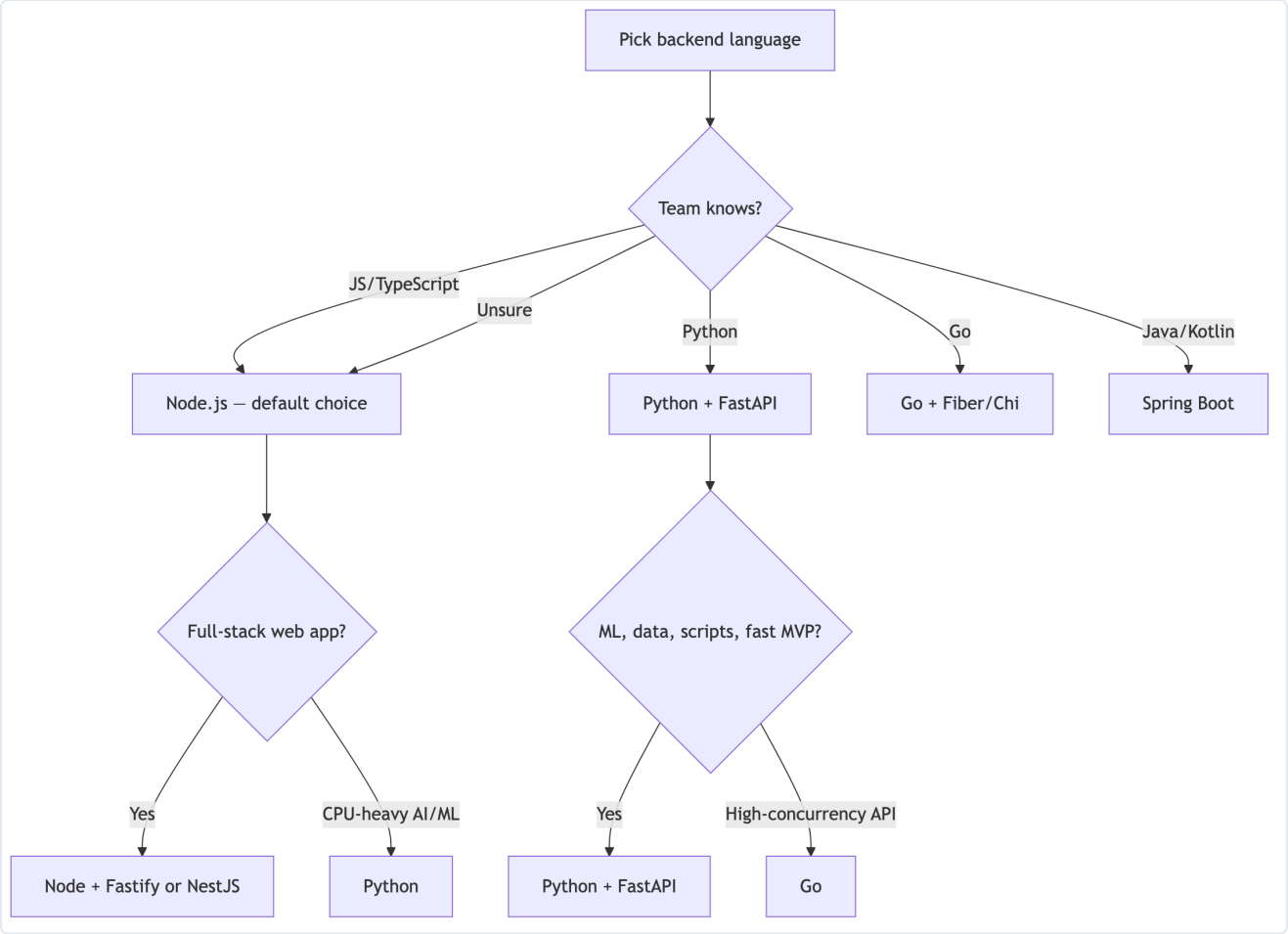


Figure 10 — 2.2 Backend language decision tree

Language	Choose when	Speed to ship	Hiring
Node.js / TypeScript	Web APIs, SaaS, full-stack TS team	⭐⭐⭐⭐⭐	Easy
Python	ML/AI, data pipelines, scripting	⭐⭐⭐⭐⭐	Easy
Go	High-throughput APIs, CLIs, infra tools	⭐⭐⭐⭐	Medium
Java / Kotlin	Enterprise, regulated, Spring shops	⭐⭐⭐	Medium

> **Default for 90% of freshers:** Node.js + TypeScript. One language front + back, massive ecosystem.

2.3 Frontend framework decision tree

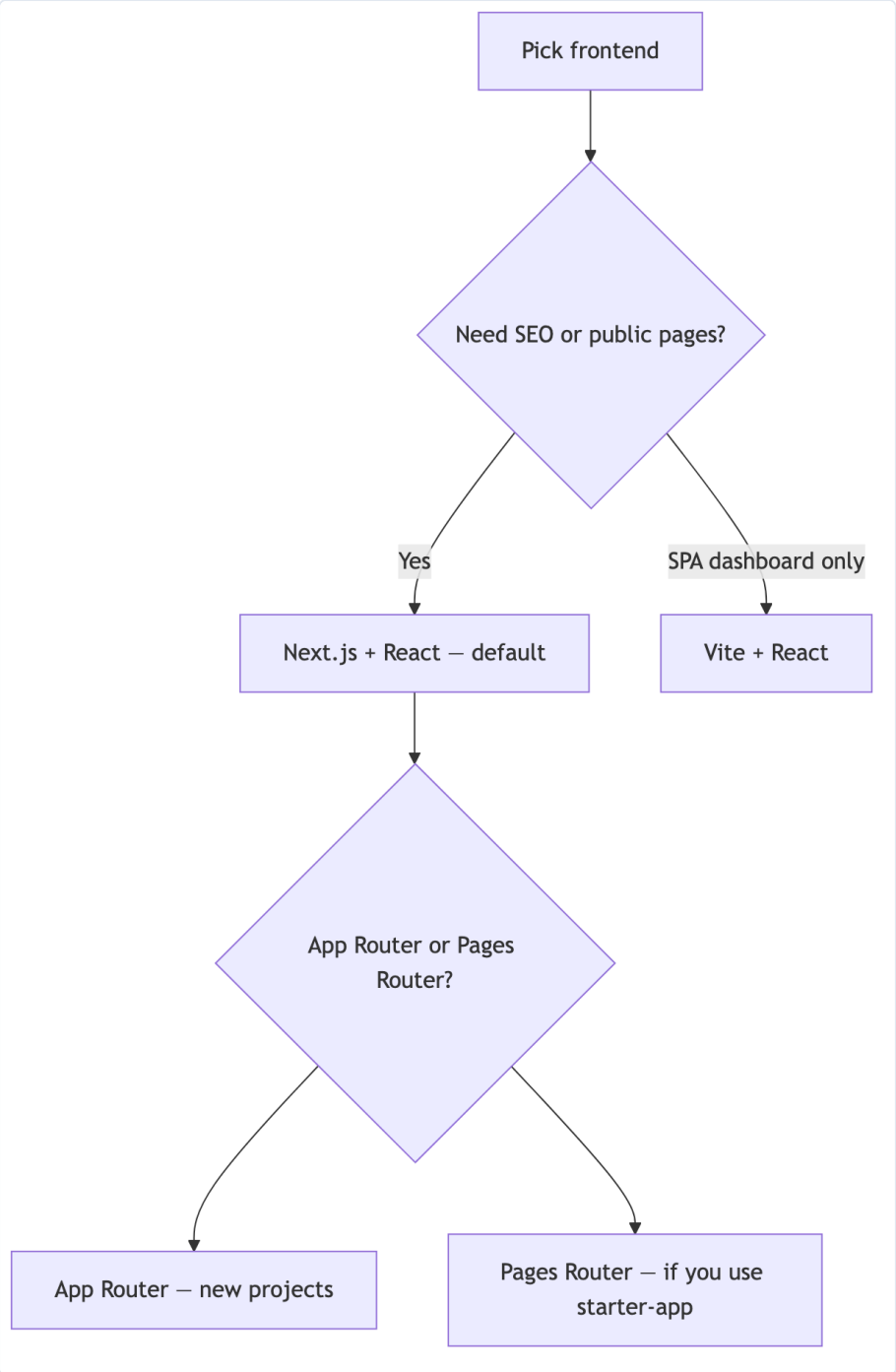


Figure 11 — 2.3 Frontend framework decision tree

Framework	Choose when
Next.js + React	Public site, SaaS, SEO, most jobs
Vite + React	Internal admin tool, no SEO needed
SvelteKit	Performance-first, tiny bundle

2.4 Database decision tree

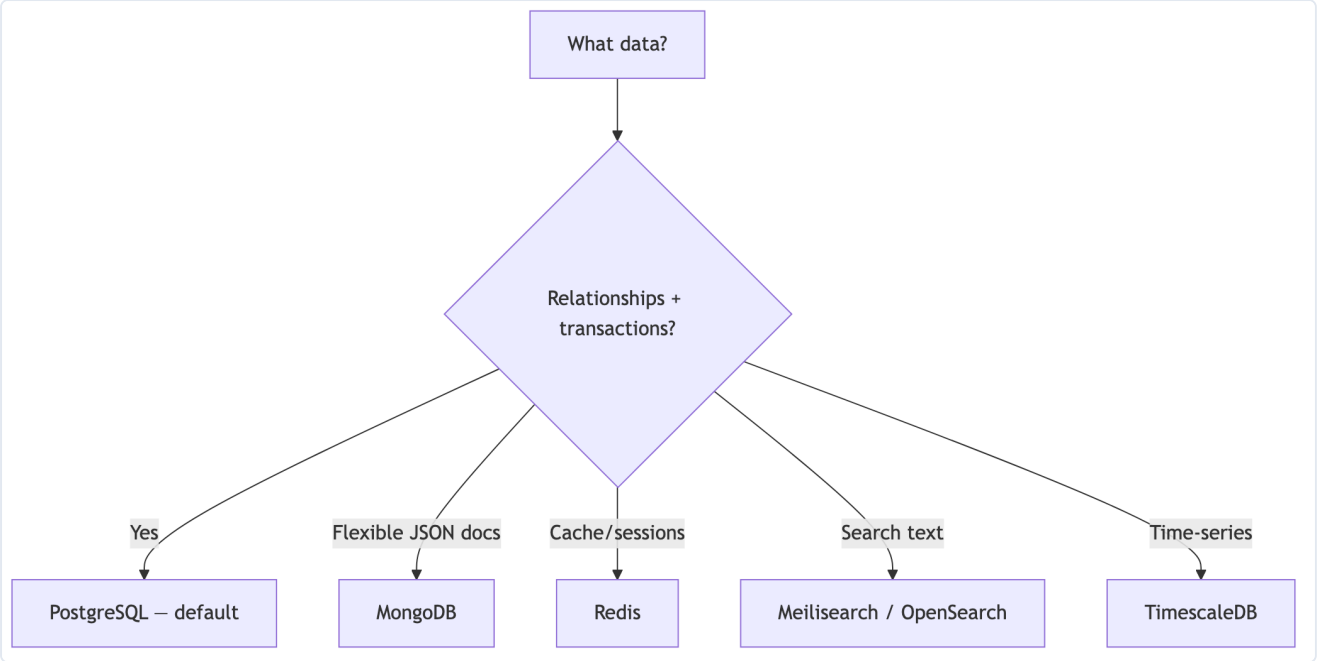


Figure 12 — 2.4 Database decision tree

Store	Use for	When to avoid
PostgreSQL	Users, orders, any relational data	Never avoid — this is your default
Redis	Cache, sessions, rate limits, queues	Don't use as primary source of truth
MongoDB	Flexible documents, catalogs	If data has relationships, use Postgres

2.5 The 2026 production stack (opinionated defaults)

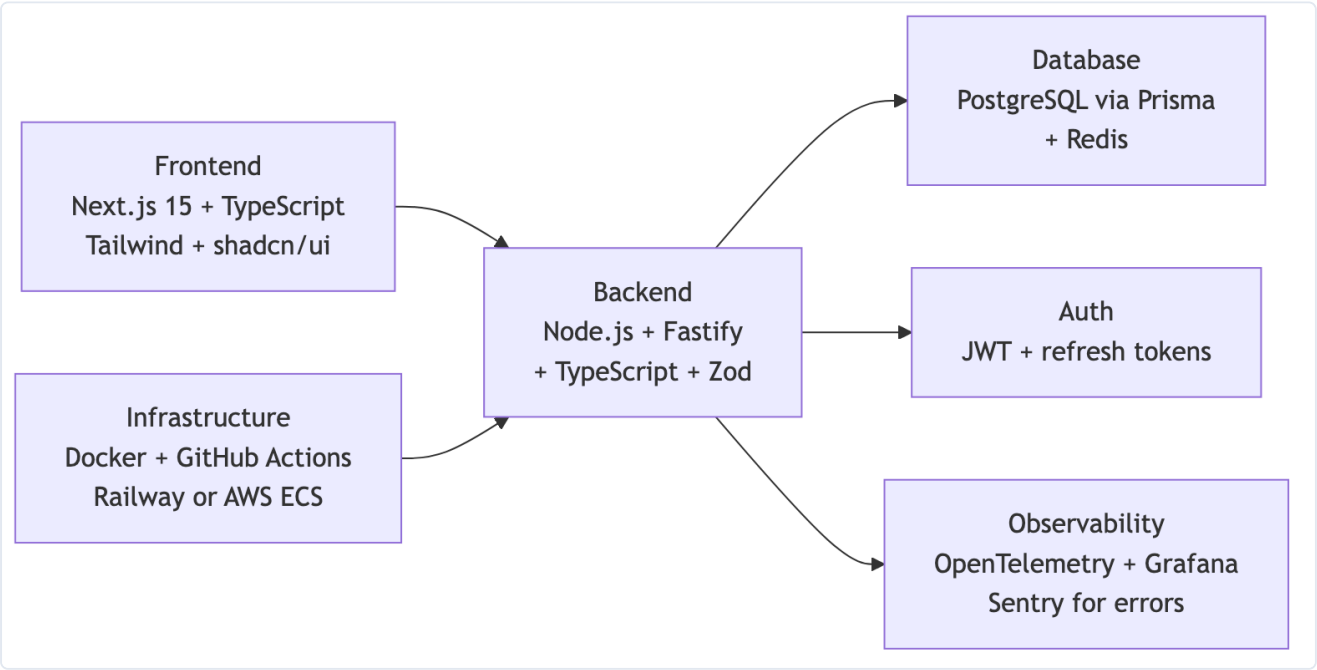


Figure 13 — 2.5 The 2026 production stack (opinionated defaults)

Layer	Choice	Why
Frontend	Next.js 15 + TypeScript	SSR, SEO, App Router, largest job market
Styling	Tailwind CSS + shadcn/ui	Fast UI, consistent design system
Server state	TanStack Query v5	Cache, refetch, loading/error states
Client state	Zustand	Simple, no boilerplate
Backend	Fastify + TypeScript	Fast, lightweight, plugin system
Validation	Zod	Runtime + compile-time, shared front/back
ORM	Prisma	Type-safe queries, automatic migrations
Database	PostgreSQL 16	Reliable, feature-rich, battle-tested
Cache	Redis 7	Sessions, cache, rate limits
Auth	JWT + refresh tokens	Stateless, scalable
Testing	Vitest + Playwright	Fast unit + real browser E2E
CI/CD	GitHub Actions	Integrated, free for public repos
Deploy	Railway / Fly.io / AWS ECS	Managed, scales when needed
Monitoring	OpenTelemetry + Grafana	Industry standard
Errors	Sentry	Best error tracking with stack traces

```
# Frontend — install the key packages
npm install @tanstack/react-query zustand
npm install react-hook-form zod @hookform/resolvers

# Backend — install the key packages
npm install fastify @fastify/cors @fastify/helmet
npm install jsonwebtoken   # use jsonwebtoken directly — NOT @fastify/jwt
npm install @prisma/client zod bcrypt jsonwebtoken ioredis pino
npm install -D prisma vitest typescript tsx
```

PART 3 — Frontend Engineering

3.1 Next.js rendering — which mode?

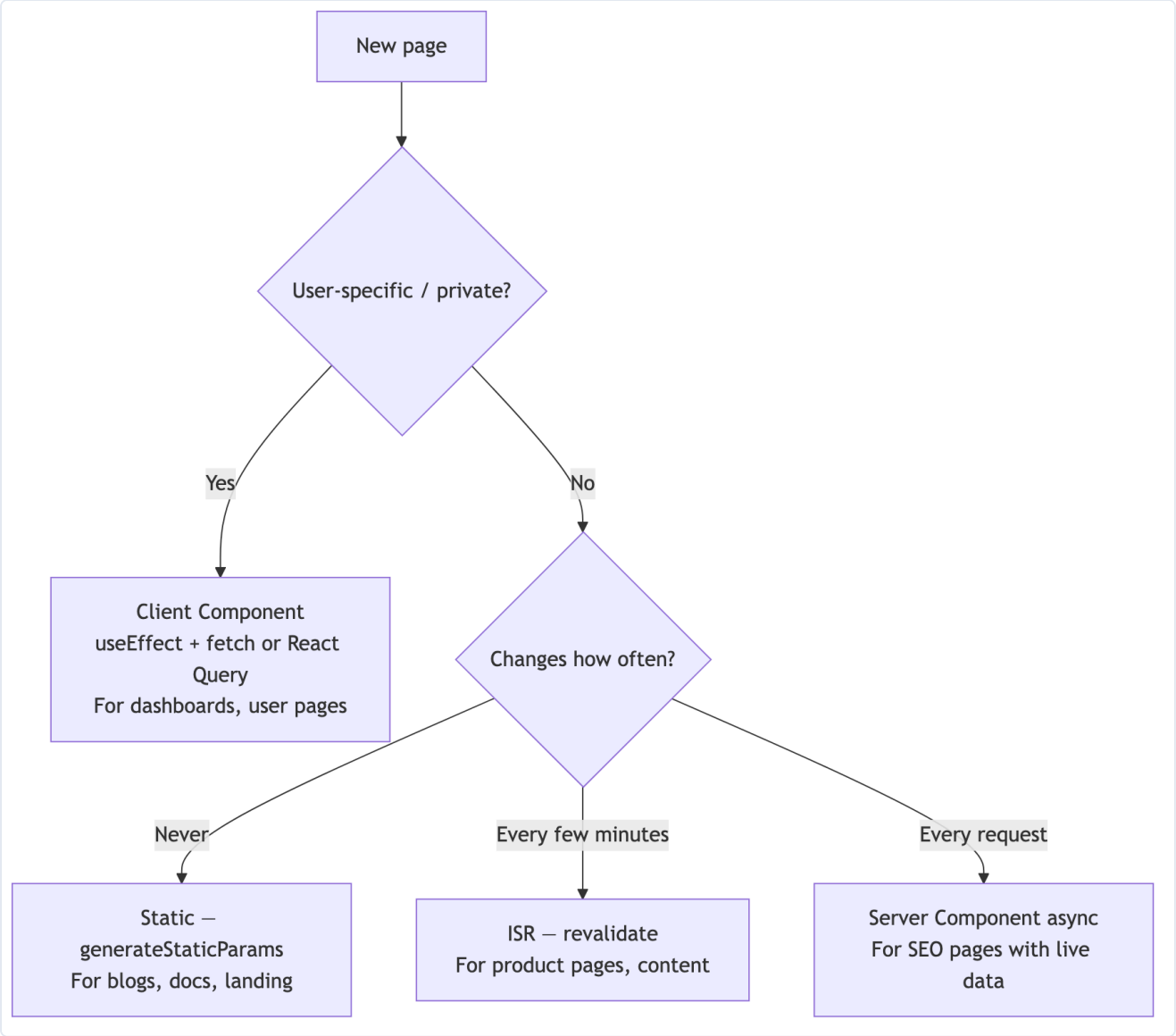


Figure 14 — 3.1 Next.js rendering — which mode?

> **Rule:** All components are Server Components by default in Next.js App Router. Add `'use client'` only when you need `useState`, `useEffect`, or event handlers.

3.2 Server Component vs Client Component

```
// ✅ Server Component — runs on server, zero JS sent to browser
// src/app/products/page.tsx
async function getProducts() {
  const res = await fetch('http://localhost:4000/products', {
    next: { revalidate: 60 }, // Cache and refresh every 60s (ISR)
  })
  if (!res.ok) throw new Error('Failed to fetch products')
  return res.json()
}

export default async function ProductsPage() {
  const products = await getProducts()
  return (
    <div className="grid grid-cols-3 gap-4 p-8">
      {products.map((p: { id: string; name: string; price: number }) => (
        <ProductCard key={p.id} product={p} />
      ))}
    </div>
  )
}
```

```
// ✅ Client Component — handles user interactions
// src/components/product-card.tsx
'use client'

import { useState } from 'react'
import { Button } from '@/components/ui/button'

interface Product {
  id: string
  name: string
  price: number
}

export function ProductCard({ product }: { product: Product }) {
  const [added, setAdded] = useState(false)

  const addToCart = async () => {
    await fetch('/api/cart', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ productId: product.id }),
    })
    setAdded(true)
  }

  return (
    <div className="border rounded-xl p-4 shadow-sm hover:shadow-md transition-shadow">
      <h3 className="font-semibold text-lg">{product.name}</h3>
      <p className="text-slate-600 mt-1">${product.price}</p>
      <Button
        onClick={addToCart}
        disabled={added}
        className="mt-3 w-full"
        variant={added ? 'outline' : 'default'}
      >
        {added ? 'Added ✓' : 'Add to Cart'}
      </Button>
    </div>
  )
}
```

3.3 State management — the full decision guide

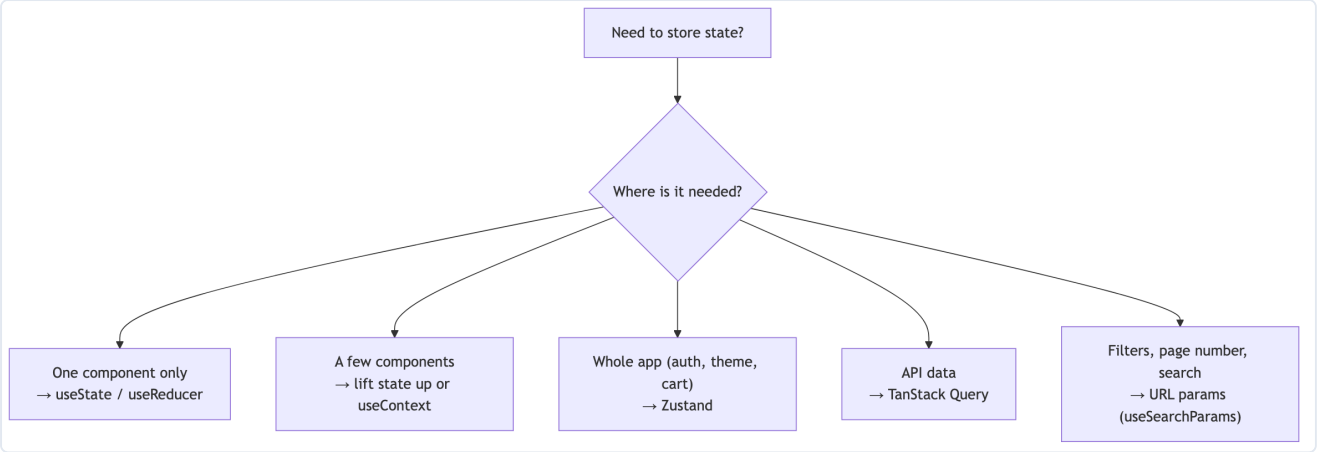


Figure 15 — 3.3 State management — the full decision guide

```
// ✅ Zustand — global state (auth, theme, shopping cart)
// src/lib/store.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

interface User {
  id: string
  email: string
  name: string
}

interface AppStore {
  user: User | null
  accessToken: string | null
  theme: 'light' | 'dark'
  setUser: (user: User | null, token: string | null) => void
  setTheme: (theme: 'light' | 'dark') => void
  logout: () => void
}

export const useAppStore = create<AppStore>()(
  persist(
    (set) => ({
      user: null,
      accessToken: null,
      theme: 'light',
      setUser: (user, accessToken) => set({ user, accessToken }),
      setTheme: (theme) => set({ theme }),
      logout: () => set({ user: null, accessToken: null }),
    }),
    { name: 'app-storage' } // persists to localStorage
  )
)
```

```tsx
// ✅ TanStack Query — server state (complete pattern)
// src/hooks/use-todos.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { useAppStore } from '@/lib/store'

const API = process.env.NEXT_PUBLIC_API_URL

interface Todo {
  id: string
  title: string
  completed: boolean
  priority: 'low' | 'medium' | 'high'
  createdAt: string
}

// Fetch all todos
export function useTodos() {
  const accessToken = useAppStore((s) => s.accessToken)

  return useQuery({
    queryKey: ['todos'],
    queryFn: async (): Promise<Todo[]> => {
      const res = await fetch(`${API}/todos`, {
        headers: { Authorization: `Bearer ${accessToken}` },
      })
      if (!res.ok) throw new Error('Failed to fetch todos')
      return res.json()
    },
    enabled: !!accessToken, // only run if logged in
    staleTime: 30_000,       // consider data fresh for 30 seconds
  })
}

// Create a todo
export function useCreateTodo() {
  const queryClient = useQueryClient()
  const accessToken = useAppStore((s) => s.accessToken)

  return useMutation({
    mutationFn: async (data: { title: string; priority: string }): Promise<Todo> => {
      const res = await fetch(`${API}/todos`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          Authorization: `Bearer ${accessToken}`,
        },
        body: JSON.stringify(data),
      })
      if (!res.ok) {
        const err = await res.json()
        throw new Error(err.error || 'Failed to create todo')
      }
      return res.json()
    },
    onSuccess: () => {
      // Invalidate the todos list so it refetches
      queryClient.invalidateQueries({ queryKey: ['todos'] })
    },
  })
}

// Toggle todo completion
export function useToggleTodo() {
  const queryClient = useQueryClient()
  const accessToken = useAppStore((s) => s.accessToken)

  return useMutation({
    mutationFn: async ({ id, completed }: { id: string; completed: boolean }): Promise<Todo> => {
      const res = await fetch(`${API}/todos/${id}`, {
        method: 'PATCH',
        headers: {
          'Content-Type': 'application/json',
          Authorization: `Bearer ${accessToken}`,
        },
        body: JSON.stringify({ completed }),
      })
      if (!res.ok) throw new Error('Failed to update todo')
      return res.json()
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['todos'] })
    },
  })
}
```

3.4 Forms — the correct pattern

```
// ✅ React Hook Form + Zod — complete working form
// src/components/create-todo-form.tsx
'use client'

import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { useCreateTodo } from '@/hooks/use-todos'

const todoSchema = z.object({
  title: z.string()
    .min(1, 'Title is required')
    .max(200, 'Title must be under 200 characters'),
  priority: z.enum(['low', 'medium', 'high']).default('medium'),
})

type TodoFormValues = z.infer<typeof todoSchema>

export function CreateTodoForm() {
  const { mutate: createTodo, isPending, error } = useCreateTodo()

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors },
  } = useForm<TodoFormValues>({
    resolver: zodResolver(todoSchema),
    defaultValues: { priority: 'medium' },
  })

  const onSubmit = (data: TodoFormValues) => {
    createTodo(data, {
      onSuccess: () => reset(),
    })
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="space-y-3">
      {/* Server error */}
      {error && (
        <p className="text-sm text-red-600 bg-red-50 border border-red-200 rounded p-2">
          {error.message}
        </p>
      )}

      <div className="flex gap-2">
        <div className="flex-1">
          <input
            {...register('title')}
            placeholder="What needs to be done?"
            className="w-full border rounded-lg px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
          />
          {errors.title && (
            <p className="text-sm text-red-500 mt-1">{errors.title.message}</p>
          )}
        </div>

        <select
          {...register('priority')}
          className="border rounded-lg px-3 py-2 bg-white"
        >
          <option value="low">Low</option>
          <option value="medium">Medium</option>
          <option value="high">High</option>
        </select>

        <button
          type="submit"
          disabled={isPending}
          className="bg-blue-600 text-white px-4 py-2 rounded-lg hover:bg-blue-700 disabled:opacity-50 transition-colors"
        >
          {isPending ? 'Adding...' : 'Add Todo'}
        </button>
      </div>
    </form>
  )
}
```

3.5 QueryClient setup (required in _app.tsx or layout.tsx)

```
// src/app/layout.tsx (App Router)
'use client'

import { useState } from 'react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { ReactQueryDevtools } from '@tanstack/react-query-devtools'

export default function RootLayout({ children }: { children: React.ReactNode }) {
  // useState ensures each request gets its own QueryClient in SSR
  const [queryClient] = useState(() => new QueryClient({
    defaultOptions: {
      queries: {
        retry: 1,
        staleTime: 30_000,
      },
    },
  }))

  return (
    <html lang="en">
      <body>
        <QueryClientProvider client={queryClient}>
          {children}
          <ReactQueryDevtools initialIsOpen={false} />
        </QueryClientProvider>
      </body>
    </html>
  )
}
```

3.6 Frontend checklist

- □ Every data view has: **loading state**, **error state**, **empty state**
- □ No API keys or secrets in client bundle (`NEXT_PUBLIC_` is public — only use for URLs)
- □ API base URL from env variable, never hardcoded
- □ Images use `next/image` (auto-optimized)
- □ Forms show field-level validation errors
- □ Buttons disabled during loading (prevent double-submit)
- □ Accessibility: labels on inputs, tab order works

3.7 Frontend security (quick)

Risk	Fix
XSS	React escapes by default. Never use `dangerouslySet-InnerHTML`
Token in localStorage	Prefer httpOnly cookies for refresh tokens
CSRF	`SameSite=Lax` on cookies
Secret leaks	`NEXT_PUBLIC_` prefix only for truly public values

PART 4 — Backend Engineering

4.1 Backend architecture — the layers

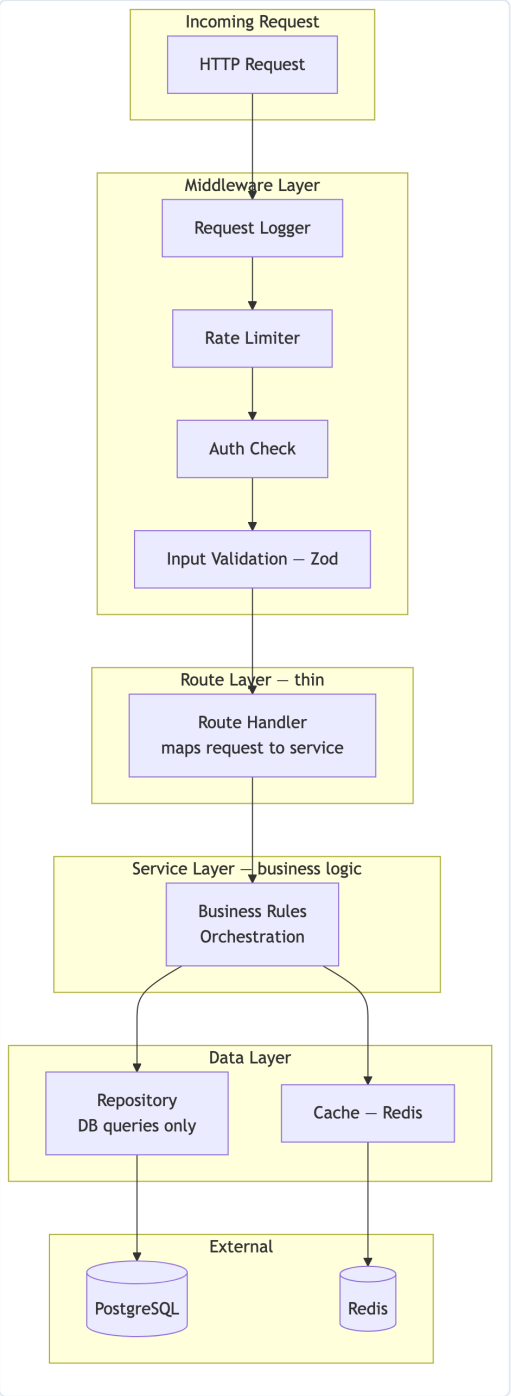


Figure 16 — 4.1 Backend architecture — the layers

The golden rule:

- **Routes** are thin — just call a service and return the result
- **Services** contain all business logic — no HTTP, no direct DB
- **Repositories** contain all DB queries — no business logic

4.2 Complete Fastify server setup

```
// api/src/index.ts
import Fastify from 'fastify'
import cors from '@fastify/cors'
import helmet from '@fastify/helmet'
import { authRoutes } from './routes/auth'
import { todoRoutes } from './routes/todos'
import { authenticate } from './middleware/auth'
import { prisma } from './lib/prisma'
import { logger } from './lib/logger'

const server = Fastify({ logger: true })

// Register plugins
await server.register(helmet)  // Security headers

await server.register(cors, {
  origin: process.env.ALLOWED_ORIGINS?.split(',') ?? ['http://localhost:3000'],
  credentials: true,
})

// Health check — always have this, no auth
server.get('/health', async () => ({
  status: 'ok',
  timestamp: new Date().toISOString(),
  db: await prisma.$queryRaw`SELECT 1`.then(() => 'ok').catch(() => 'error'),
}))

// Routes
await server.register(authRoutes, { prefix: '/auth' })
await server.register(todoRoutes, { prefix: '/todos' })

// Global error handler — no stack traces to clients in production
server.setErrorHandler((error, request, reply) => {
  logger.error({ error, url: request.url, method: request.method }, 'Request error')

  // Zod validation error
  if (error.name === 'ZodError') {
    return reply.status(400).send({
      error: 'Validation failed',
      code: 'VALIDATION_ERROR',
      details: JSON.parse(error.message),
    })
  }

  const statusCode = error.statusCode ?? 500
  return reply.status(statusCode).send({
    error: statusCode === 500 ? 'Internal server error' : error.message,
    code: statusCode === 500 ? 'INTERNAL_ERROR' : 'REQUEST_ERROR',
  })
})

// Graceful shutdown — always handle SIGTERM
const shutdown = async () => {
  await server.close()
  await prisma.$disconnect()
  process.exit(0)
}
process.on('SIGTERM', shutdown)
process.on('SIGINT', shutdown)

// Start
try {
  await server.listen({ port: Number(process.env.PORT) || 4000, host: '0.0.0.0' })
  logger.info('Server running on port 4000')
} catch (err) {
  server.log.error(err)
  process.exit(1)
}
```

4.3 App factory — the key to testable code

> **Why this matters:** The `index.ts` above starts the server immediately. Tests can't use that — they need to create the app, run assertions, then close it cleanly. The solution is to split your code into two files: `app.ts` (creates and configures Fastify) and `index.ts` (calls `buildApp()` then listens).

```
// api/src/app.ts  ← create this file
import Fastify, { FastifyInstance } from 'fastify'
import cors from '@fastify/cors'
import helmet from '@fastify/helmet'
import { authRoutes } from './routes/auth'
import { todoRoutes } from './routes/todos'
import { authenticate } from './middleware/auth'
import { prisma } from './lib/prisma'

export async function buildApp(): Promise<FastifyInstance> {
  const app = Fastify({
    logger: process.env.NODE_ENV !== 'test', // silence logs during tests
  })

  await app.register(helmet)

  await app.register(cors, {
    origin: process.env.ALLOWED_ORIGINS?.split(',') ?? ['http://localhost:3000'],
    credentials: true,
  })

  // Decorate with authenticate so routes can use it as preHandler
  app.decorate('authenticate', authenticate)

  // Health check
  app.get('/health', async () => ({
    status: 'ok',
    timestamp: new Date().toISOString(),
    db: await prisma.$queryRaw`SELECT 1`.then(() => 'ok').catch(() => 'error'),
  }))

  await app.register(authRoutes, { prefix: '/auth' })
  await app.register(todoRoutes, { prefix: '/todos' })

  app.setErrorHandler((error, request, reply) => {
    if (error.name === 'ZodError') {
      return reply.status(400).send({
        error: 'Validation failed',
        code: 'VALIDATION_ERROR',
        details: JSON.parse(error.message),
      })
    }
    const statusCode = error.statusCode ?? 500
    return reply.status(statusCode).send({
      error: statusCode === 500 ? 'Internal server error' : error.message,
      code: statusCode === 500 ? 'INTERNAL_ERROR' : 'REQUEST_ERROR',
    })
  })

  return app
}
```

```
// api/src/index.ts  ← now just starts the server
import { buildApp } from './app'
import { prisma } from './lib/prisma'

const app = await buildApp()

const shutdown = async () => {
  await app.close()
  await prisma.$disconnect()
  process.exit(0)
}
process.on('SIGTERM', shutdown)
process.on('SIGINT', shutdown)

try {
  await app.listen({ port: Number(process.env.PORT) || 4000, host: '0.0.0.0' })
} catch (err) {
  app.log.error(err)
  process.exit(1)
}
```

> **The rule:** `app.ts` = build and configure. `index.ts` = listen. Tests import `buildApp` from `app.ts` — they never touch `index.ts`.

4.4 Complete CRUD routes

```
// api/src/routes/todos.ts
import { FastifyPluginAsync } from 'fastify'
import { z } from 'zod'
import { TodoService } from '../services/todo.service'
import { authenticate } from '../middleware/auth'

const createTodoSchema = z.object({
  title: z.string().min(1, 'Title required').max(200),
  priority: z.enum(['low', 'medium', 'high']).default('medium'),
})

const updateTodoSchema = z.object({
  title: z.string().min(1).max(200).optional(),
  completed: z.boolean().optional(),
  priority: z.enum(['low', 'medium', 'high']).optional(),
})

export const todoRoutes: FastifyPluginAsync = async (fastify) => {
  const todoService = new TodoService()

  // GET /todos — list all todos for current user
  fastify.get('/', { preHandler: [authenticate] }, async (request, reply) => {
    const todos = await todoService.findAll(request.user.userId)
    return reply.send(todos)
  })

  // GET /todos/:id
  fastify.get('/:id', { preHandler: [authenticate] }, async (request, reply) => {
    const { id } = request.params as { id: string }
    const todo = await todoService.findById(id, request.user.userId)
    if (!todo) return reply.status(404).send({ error: 'Todo not found' })
    return reply.send(todo)
  })

  // POST /todos
  fastify.post('/', { preHandler: [authenticate] }, async (request, reply) => {
    const body = createTodoSchema.parse(request.body)
    const todo = await todoService.create({ ...body, userId: request.user.userId })
    return reply.status(201).send(todo)
  })

  // PATCH /todos/:id
  fastify.patch('/:id', { preHandler: [authenticate] }, async (request, reply) => {
    const { id } = request.params as { id: string }
    const body = updateTodoSchema.parse(request.body)
    const todo = await todoService.update(id, request.user.userId, body)
    if (!todo) return reply.status(404).send({ error: 'Todo not found' })
    return reply.send(todo)
  })

  // DELETE /todos/:id
  fastify.delete('/:id', { preHandler: [authenticate] }, async (request, reply) => {
    await todoService.delete(id, request.user.userId)
    return reply.status(204).send()
  })
}
```

4.4 Service layer — business logic

```
// api/src/services/todo.service.ts
import { TodoRepository } from '../repositories/todo.repository'
import { cache } from '../lib/redis'

export class TodoService {
  private repo = new TodoRepository()

  async findAll(userId: string) {
    const cacheKey = `todos:${userId}`
    const cached = await cache.get<Todo[]>(cacheKey)
    if (cached) return cached

    const todos = await this.repo.findByUser(userId)
    await cache.set(cacheKey, todos, 60) // cache 60 seconds
    return todos
  }

  async findById(id: string, userId: string) {
    return this.repo.findOne(id, userId)
  }

  async create(data: { title: string; priority: string; userId: string }) {
    const todo = await this.repo.create(data)
    await cache.del(`todos:${data.userId}`) // invalidate list cache
    return todo
  }

  async update(id: string, userId: string, data: Partial<{ title: string; completed: boolean; priority: string }>) {
    const todo = await this.repo.update(id, userId, data)
    if (todo) await cache.del(`todos:${userId}`)
    return todo
  }

  async delete(id: string, userId: string) {
    await this.repo.delete(id, userId)
    await cache.del(`todos:${userId}`)
  }
}
```

4.5 Repository layer — database queries

```
// api/src/repositories/todo.repository.ts
import { prisma } from '../lib/prisma'

export class TodoRepository {
  findByUser(userId: string) {
    return prisma.todo.findMany({
      where: { userId },
      orderBy: { createdAt: 'desc' },
    })
  }

  findOne(id: string, userId: string) {
    return prisma.todo.findFirst({
      where: { id, userId }, // ALWAYS filter by userId — security!
    })
  }

  create(data: { title: string; priority: string; userId: string }) {
    return prisma.todo.create({ data })
  }

  update(id: string, userId: string, data: object) {
    return prisma.todo.update({
      where: { id, userId },
      data,
    })
  }

  delete(id: string, userId: string) {
    return prisma.todo.delete({
      where: { id, userId },
    })
  }
}
```

4.6 HTTP status codes — use these right

Status	Meaning	When
`200 OK`	Success	GET, PATCH, PUT worked
`201 Created`	Resource created	POST that creates a row
`204 No Content`	Success, no body	DELETE
`400 Bad Request`	Client sent bad data	Validation failed
`401 Unauthorized`	Not authenticated	No token or invalid token
`403 Forbidden`	Not authorized	Has token but no permission
`404 Not Found`	Doesn't exist	Row not found
`409 Conflict`	Duplicate	Email already registered
`429 Too Many Requests`	Rate limited	Too many requests
`500 Internal Error`	Server bug	Unexpected crash

4.7 Rate limiting

```
// api/src/middleware/rate-limit.ts
import { FastifyRequest, FastifyReply } from 'fastify'
import { redis } from '../lib/redis'

export async function rateLimit(
  request: FastifyRequest,
  reply: FastifyReply,
  options = { limit: 100, windowSeconds: 60 }
) {
  const identifier = request.headers['x-forwarded-for'] as string || request.ip
  const key = `ratelimit:${identifier}:${request.routerPath}`

  const current = await redis.incr(key)
  if (current === 1) await redis.expire(key, options.windowSeconds)

  reply.header('X-RateLimit-Limit', options.limit)
  reply.header('X-RateLimit-Remaining', Math.max(0, options.limit - current))

  if (current > options.limit) {
    return reply.status(429).send({
      error: 'Too many requests. Please slow down.',
      code: 'RATE_LIMIT_EXCEEDED',
    })
  }
}

// Stricter limit for auth routes (prevent brute force)
export const authRateLimit = (req: FastifyRequest, reply: FastifyReply) =>
  rateLimit(req, reply, { limit: 10, windowSeconds: 60 })
```

4.8 Background jobs (send emails without blocking the request)

```
// api/src/lib/queue.ts
import { Queue, Worker } from 'bullmq'
import { redis } from './redis'

export const emailQueue = new Queue('email', { connection: redis })

// Add a job (non-blocking — returns immediately)
export async function queueWelcomeEmail(userId: string, email: string) {
  await emailQueue.add(
    'welcome-email',
    { userId, email },
    {
      attempts: 3,
      backoff: { type: 'exponential', delay: 2000 },
      removeOnComplete: true,
    }
  )
}

// Worker processes jobs in the background
const emailWorker = new Worker(
  'email',
  async (job) => {
    if (job.name === 'welcome-email') {
      const { userId, email } = job.data
      // await sendWelcomeEmail(email, userId)
      console.log(`Sending welcome email to ${email}`)
    }
  },
  { connection: redis }
)

emailWorker.on('failed', (job, err) => {
  console.error(`Job ${job?.id} failed:`, err.message)
})
```

4.9 Backend checklist

- ☐ `GET /health` — no auth, checks DB connectivity
- ☐ Global error handler — no raw stack traces to clients
- ☐ CORS configured for frontend origin only
- ☐ All write routes validate with Zod
- ☐ DB uses connection pool (Prisma handles this automatically)
- ☐ Graceful shutdown on SIGTERM
- ☐ JSON structured logs
- ☐ Rate limiting on auth endpoints

PART 5 — Database Engineering

5.1 Prisma schema — define your data

```
// api/prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id           String    @id @default(cuid())
  email        String    @unique
  name         String?
  passwordHash String
  role         Role      @default(USER)
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt

  todos        Todo[]
  sessions     Session[]

  @@index([email])
}

model Todo {
  id        String    @id @default(cuid())
  title     String
  completed Boolean   @default(false)
  priority  Priority  @default(MEDIUM)
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt

  userId    String
  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([userId, completed])  -- compound index for common filter
}

model Session {
  id           String   @id @default(cuid())
  refreshToken String   @unique
  userId       String
  user         User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  expiresAt    DateTime
  createdAt    DateTime @default(now())

  @@index([userId])
  @@index([refreshToken])
}

enum Role {
  USER
  ADMIN
}

enum Priority {
  LOW
  MEDIUM
  HIGH
}
```

```
# Run migration in development (creates migration file + applies it)
npx prisma migrate dev --name init

# Apply migrations in production (CI/CD)
npx prisma migrate deploy

# View your data in browser
npx prisma studio

# Regenerate the Prisma client after schema changes
npx prisma generate
```

5.2 Prisma client setup

```
// api/src/lib/prisma.ts
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient({
  log: process.env.NODE_ENV === 'development'
    ? ['query', 'warn', 'error']
    : ['warn', 'error'],
})

// Log queries slower than 1 second
prisma.$on('query' as never, (e: any) => {
  if (e.duration > 1000) {
    console.warn(`Slow query (${e.duration}ms):`, e.query.substring(0, 200))
  }
})

export { prisma }
```

5.3 Indexing — speed up your queries

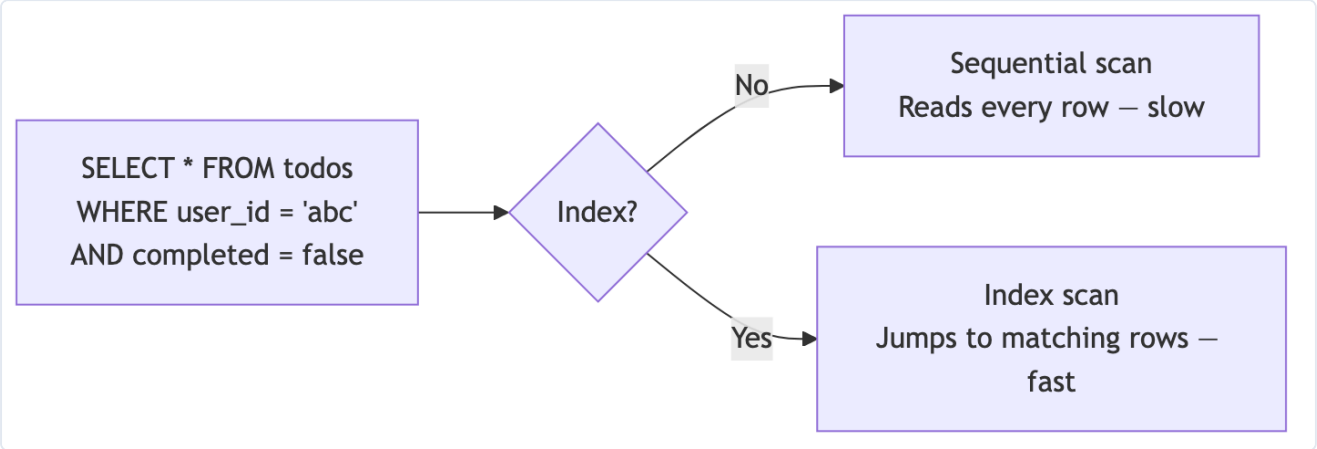


Figure 17 — 5.3 Indexing — speed up your queries

> **Rule:** Add an index on any column you `WHERE`, `JOIN ON`, or `ORDER BY`.

```
-- Add these after creating your tables
CREATE INDEX idx_todos_user_id ON todos(user_id);
CREATE INDEX idx_todos_user_completed ON todos(user_id, completed);
CREATE INDEX idx_users_email ON users(email);

-- Find slow queries (run in psql)
EXPLAIN ANALYZE SELECT * FROM todos WHERE user_id = 'abc' AND completed = false;
```

> **Watch out:** Don't index every column — indexes slow down writes. Index only columns you actually query.

5.4 Fix the N+1 query problem

```
// ❌ N+1 problem — runs 1 query for users + N queries for todos
const users = await prisma.user.findMany()
for (const user of users) {
  user.todos = await prisma.todo.findMany({ where: { userId: user.id } })
  // This makes 1 DB trip per user!
}

// ✅ Fix with include — runs just 1 query total
const users = await prisma.user.findMany({
  include: { todos: true },
})

// ✅ Fix with JOIN in raw SQL
const users = await prisma.$queryRaw`
  SELECT u.*, json_agg(t.*) as todos
  FROM users u
  LEFT JOIN todos t ON t.user_id = u.id
  GROUP BY u.id
`
```

5.5 Migrations — never break production

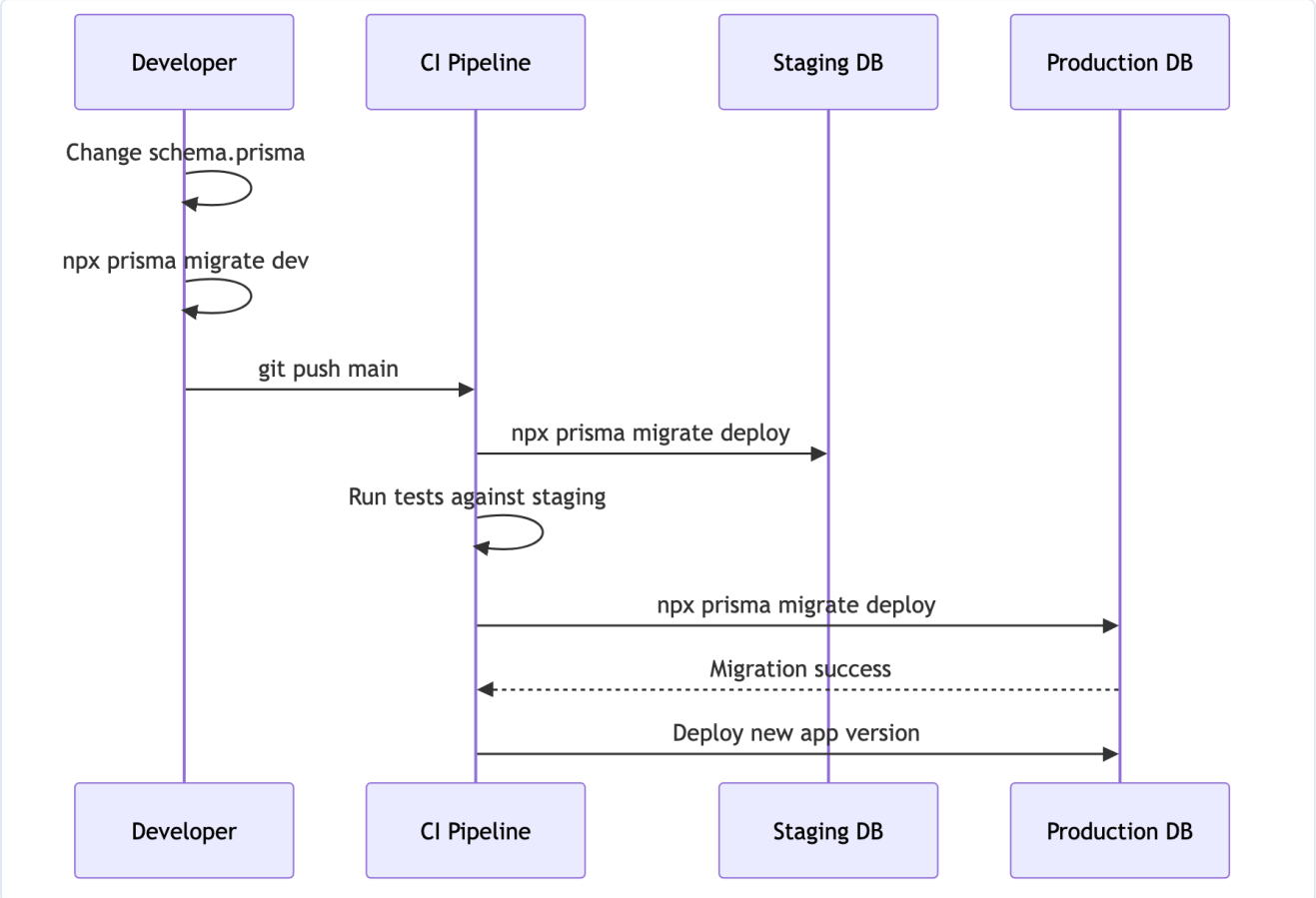


Figure 18 — 5.5 Migrations — never break production

Zero-downtime migration rules:

- Add columns as `nullable` first, then backfill, then add constraint
- Never rename columns directly — add new, copy data, remove old
- Add indexes with `CREATE INDEX CONCURRENTLY` (no table lock)
- Run `prisma migrate deploy` BEFORE deploying the new app version

5.6 Redis caching

```
// api/src/lib/redis.ts
import Redis from 'ioredis'

export const redis = new Redis(process.env.REDIS_URL!, {
  maxRetriesPerRequest: 3,
  retryStrategy: (times) => Math.min(times * 50, 2000),
})

redis.on('error', (err) => console.error('Redis error:', err.message))

export const cache = {
  async get<T>(key: string): Promise<T | null> {
    const val = await redis.get(key)
    return val ? JSON.parse(val) : null
  },

  async set(key: string, value: unknown, ttlSeconds = 300) {
    await redis.setex(key, ttlSeconds, JSON.stringify(value))
  },

  async del(key: string) {
    await redis.del(key)
  },

  async invalidatePattern(pattern: string) {
    const keys = await redis.keys(pattern)
    if (keys.length > 0) await redis.del(...keys)
  },
}
```

Data	Cache?	TTL	Reason
User profile	Yes	5-10 min	Rarely changes
Product list	Yes	1-5 min	High read traffic
Shopping cart	No	—	Must be real-time
Auth sessions	Yes (Redis)	= token expiry	Fast lookup
Search results	Yes	1 min	Expensive queries

5.7 Database checklist

- □ Indexes on foreign keys and frequent `WHERE` columns
- □ Automated backups enabled + restore tested once
- □ No `SELECT *` in hot paths — select specific columns
- □ Connection pool configured (Prisma: default 10, set `connection_limit`)
- □ Migrations run before app deploys
- □ Slow query logging enabled

PART 6 — Auth & Security

6.1 JWT auth flow

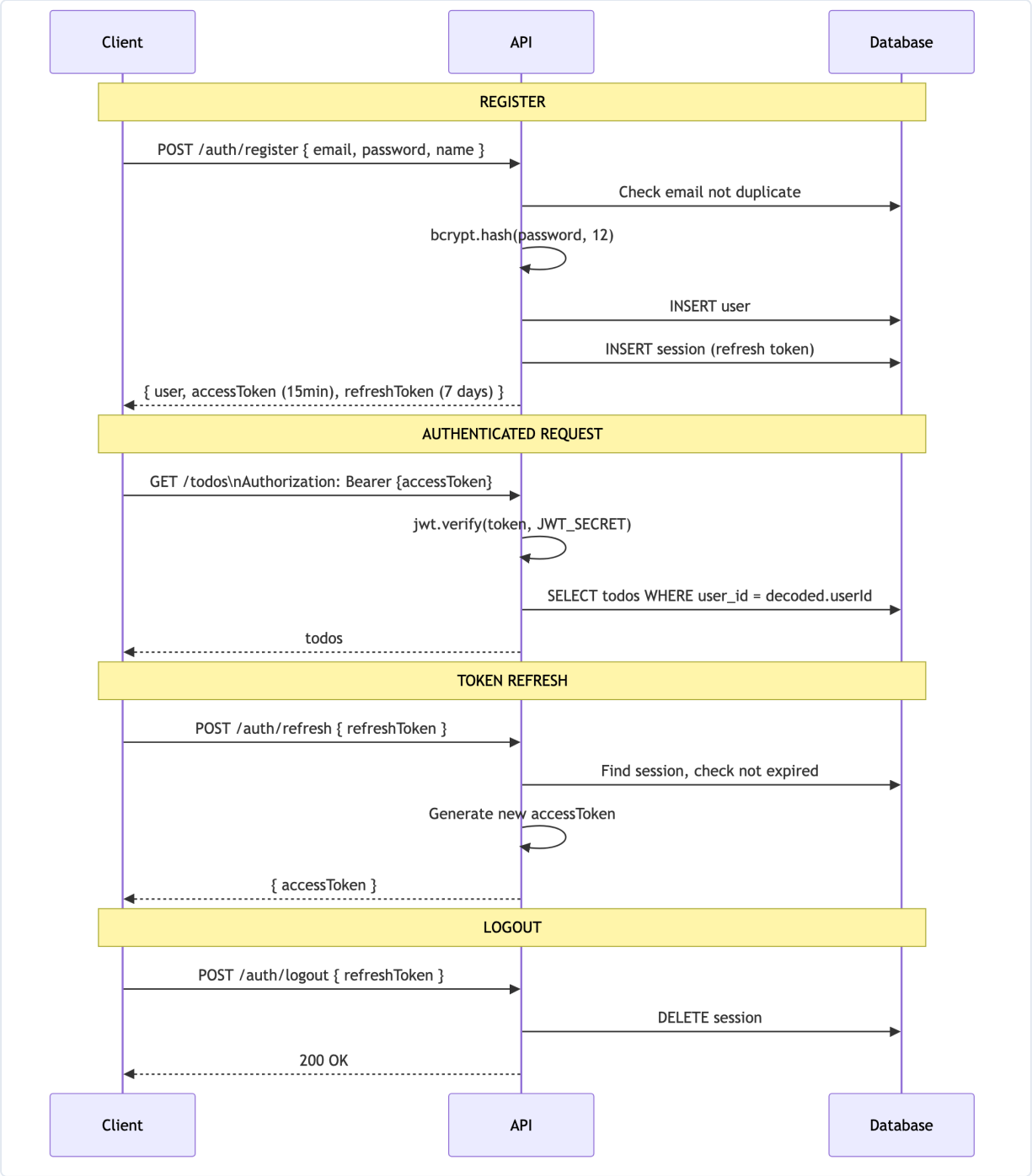


Figure 19 — 6.1 JWT auth flow

6.2 Complete auth routes

```
// api/src/routes/auth.ts
import bcrypt from 'bcrypt'
import { FastifyPluginAsync } from 'fastify'
import { z } from 'zod'
import { prisma } from '../lib/prisma'
import { generateTokens } from '../lib/jwt'
import { authenticate } from '../middleware/auth'

const registerSchema = z.object({
  email: z.string().email('Invalid email'),
  password: z.string().min(8, 'Password must be at least 8 characters'),
  name: z.string().min(1).max(100),
})

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string(),
})

export const authRoutes: FastifyPluginAsync = async (fastify) => {

  // POST /auth/register
  fastify.post('/register', async (request, reply) => {
    const { email, password, name } = registerSchema.parse(request.body)

    const existing = await prisma.user.findUnique({ where: { email } })
    if (existing) {
      return reply.status(409).send({ error: 'Email already in use', code: 'EMAIL_TAKEN' })
    }

    const passwordHash = await bcrypt.hash(password, 12)

    const user = await prisma.user.create({
      data: { email, passwordHash, name },
    })

    const { accessToken, refreshToken } = await generateTokens(user.id)

    return reply.status(201).send({
      user: { id: user.id, email: user.email, name: user.name },
      accessToken,
      refreshToken,
    })
  })

  // POST /auth/login
  fastify.post('/login', async (request, reply) => {
    const { email, password } = loginSchema.parse(request.body)

    const user = await prisma.user.findUnique({ where: { email } })

    // Constant-time comparison — don't reveal if email exists
    const validPassword = user
      ? await bcrypt.compare(password, user.passwordHash)
      : await bcrypt.compare(password, '$2b$12$invalidhashfortimingattack')

    if (!user || !validPassword) {
      return reply.status(401).send({
        error: 'Invalid email or password',
        code: 'INVALID_CREDENTIALS',
      })
    }

    const { accessToken, refreshToken } = await generateTokens(user.id)

    return reply.send({
      user: { id: user.id, email: user.email, name: user.name },
      accessToken,
      refreshToken,
    })
  })

  // POST /auth/refresh
  fastify.post('/refresh', async (request, reply) => {
    const { refreshToken } = request.body as { refreshToken: string }

    if (!refreshToken) {
      return reply.status(400).send({ error: 'Refresh token required' })
    }

    const session = await prisma.session.findUnique({
      where: { refreshToken },
      include: { user: true },
    })
```

```
    if (!session || session.expiresAt < new Date()) {
      if (session) await prisma.session.delete({ where: { id: session.id } })
      return reply.status(401).send({ error: 'Invalid or expired refresh token' })
    }

    const { accessToken } = await generateTokens(session.userId)
    return reply.send({ accessToken })
  })

  // POST /auth/logout
  fastify.post('/logout', async (request, reply) => {
    const { refreshToken } = request.body as { refreshToken: string }
    if (refreshToken) {
      await prisma.session.deleteMany({ where: { refreshToken } })
    }
    return reply.send({ message: 'Logged out' })
  })

  // GET /auth/me — protected
  fastify.get('/me', { preHandler: [authenticate] }, async (request, reply) => {
    const user = await prisma.user.findUnique({
      where: { id: request.user.userId },
      select: { id: true, email: true, name: true, role: true, createdAt: true },
    })
    if (!user) return reply.status(404).send({ error: 'User not found' })
    return reply.send(user)
  })
}
```

6.3 JWT utilities

```
// api/src/lib/jwt.ts
import jwt from 'jsonwebtoken'
import { prisma } from './prisma'

const ACCESS_SECRET = process.env.JWT_SECRET!
const REFRESH_SECRET = process.env.JWT_REFRESH_SECRET!

if (!ACCESS_SECRET || !REFRESH_SECRET) {
  throw new Error('JWT_SECRET and JWT_REFRESH_SECRET must be set')
}

export function generateAccessToken(userId: string): string {
  return jwt.sign({ userId }, ACCESS_SECRET, { expiresIn: '15m' })
}

export async function generateTokens(userId: string) {
  const accessToken = generateAccessToken(userId)
  const refreshToken = jwt.sign({ userId }, REFRESH_SECRET, { expiresIn: '7d' })

  await prisma.session.create({
    data: {
      userId,
      refreshToken,
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
    },
  })

  return { accessToken, refreshToken }
}
```

6.4 Auth middleware

```
// api/src/middleware/auth.ts
import { FastifyRequest, FastifyReply } from 'fastify'
import jwt from 'jsonwebtoken'

declare module 'fastify' {
  interface FastifyRequest {
    user: { userId: string }
  }
}

export async function authenticate(request: FastifyRequest, reply: FastifyReply) {
  const authHeader = request.headers.authorization
  if (!authHeader?.startsWith('Bearer ')) {
    return reply.status(401).send({ error: 'Authorization token required', code: 'NO_TOKEN' })
  }

  const token = authHeader.replace('Bearer ', '')

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!) as { userId: string }
    request.user = { userId: payload.userId }
  } catch (err) {
    if (err instanceof jwt.TokenExpiredError) {
      return reply.status(401).send({ error: 'Token expired — please refresh', code: 'TOKEN_EXPIRED' })
    }
    return reply.status(401).send({ error: 'Invalid token', code: 'INVALID_TOKEN' })
  }
}
```

6.5 Token storage (where to put tokens in the browser)

Token	Where to store	Why
Access token (15min)	JavaScript memory or short-lived cookie	Short lifetime, okay to lose on refresh
Refresh token (7 days)	`httpOnly Secure SameSite=Strict` cookie	Can't be read by JS — XSS protection

```
// On login, set refresh token as httpOnly cookie:
reply.setCookie('refreshToken', refreshToken, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'strict',
  maxAge: 7 * 24 * 60 * 60, // 7 days in seconds
  path: '/auth/refresh',
})
```

6.6 Security checklist (non-negotiable)

- □ HTTPS everywhere (never HTTP in production)
- □ Passwords hashed with bcrypt (rounds=12, NEVER store plain text)
- □ JWT secrets in env/secret manager, never in code
- □ `httpOnly` cookie for refresh tokens
- □ Input validation with Zod on every write endpoint
- □ Auth middleware on every private route
- □ Rate limiting on `/auth/login` and `/auth/register`
- □ Dependency scanning in CI (Dependabot or Snyk)
- □ CORS configured for your frontend origin only
- □ Helmet security headers registered
- □ SQL via Prisma (no string concatenation — no SQL injection)

PART 7 — Testing

7.1 The testing pyramid

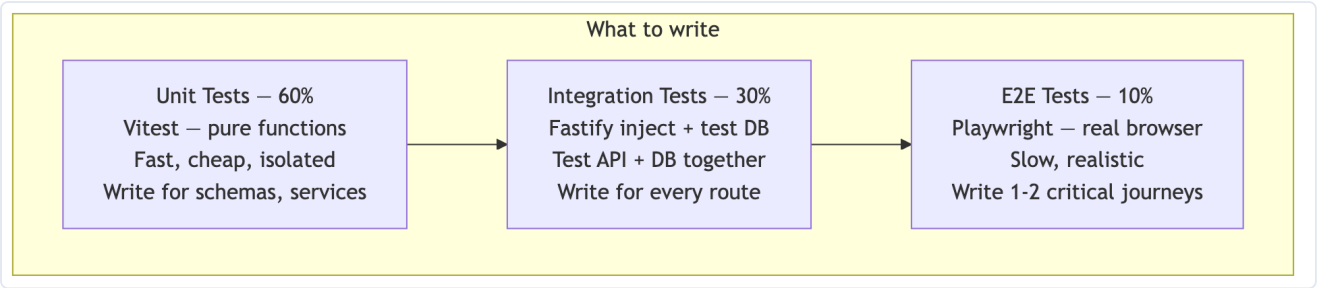


Figure 20 — 7.1 The testing pyramid

> **Start with:** unit tests for your Zod schemas (quick wins), then integration tests for each route.

7.2 Unit tests — validate your schemas

```
// api/src/tests/validation.test.ts
import { describe, it, expect } from 'vitest'
import { z } from 'zod'

const createTodoSchema = z.object({
  title: z.string().min(1, 'Title required').max(200),
  priority: z.enum(['low', 'medium', 'high']).default('medium'),
})

describe('createTodoSchema', () => {
  it('accepts valid todo', () => {
    const result = createTodoSchema.parse({ title: 'Buy milk', priority: 'high' })
    expect(result.title).toBe('Buy milk')
    expect(result.priority).toBe('high')
  })

  it('rejects empty title', () => {
    expect(() => createTodoSchema.parse({ title: '' })).toThrow()
  })

  it('rejects title over 200 chars', () => {
    expect(() => createTodoSchema.parse({ title: 'x'.repeat(201) })).toThrow()
  })

  it('defaults priority to medium', () => {
    const result = createTodoSchema.parse({ title: 'Test' })
    expect(result.priority).toBe('medium')
  })

  it('rejects invalid priority', () => {
    expect(() => createTodoSchema.parse({ title: 'Test', priority: 'critical' })).toThrow()
  })
})

const registerSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
  name: z.string().min(1),
})

describe('registerSchema', () => {
  it('accepts valid user', () => {
    expect(() => registerSchema.parse({
      email: 'user@example.com',
      password: 'secret123',
      name: 'Alice',
    })).not.toThrow()
  })

  it('rejects bad email', () => {
    expect(() => registerSchema.parse({
      email: 'notanemail',
      password: 'secret123',
      name: 'Alice',
    })).toThrow()
  })

  it('rejects short password', () => {
    expect(() => registerSchema.parse({
      email: 'user@example.com',
      password: 'short',
      name: 'Alice',
    })).toThrow()
  })
})
```

7.3 Integration tests — test your API routes

```
// api/src/tests/health.test.ts
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import { buildApp } from '../app'
import type { FastifyInstance } from 'fastify'

describe('Health endpoint', () => {
  let app: FastifyInstance

  beforeAll(async () => {
    app = await buildApp()
  })

  afterAll(async () => {
    await app.close()
  })

  it('GET /health returns 200', async () => {
    const res = await app.inject({
      method: 'GET',
      url: '/health',
    })
    expect(res.statusCode).toBe(200)
    expect(JSON.parse(res.body)).toMatchObject({ status: 'ok' })
  })
})

// api/src/tests/todos.test.ts
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import { buildApp } from '../app'
import { prisma } from '../lib/prisma'
import { generateAccessToken } from '../lib/jwt'

describe('Todos API', () => {
  let app: Awaited<ReturnType<typeof buildApp>>
  let token: string
  let userId: string

  beforeAll(async () => {
    app = await buildApp()

    // Create test user
    const user = await prisma.user.create({
      data: {
        email: `test-${Date.now()}@test.com`,
        passwordHash: 'hashed',
        name: 'Test User',
      },
    })
    userId = user.id
    token = generateAccessToken(userId)
  })

  afterAll(async () => {
    await prisma.user.delete({ where: { id: userId } })
    await app.close()
  })

  it('GET /todos returns empty array for new user', async () => {
    const res = await app.inject({
      method: 'GET',
      url: '/todos',
      headers: { Authorization: `Bearer ${token}` },
    })
    expect(res.statusCode).toBe(200)
    expect(JSON.parse(res.body)).toEqual([])
  })

  it('POST /todos creates a todo', async () => {
    const res = await app.inject({
      method: 'POST',
      url: '/todos',
      headers: {
        Authorization: `Bearer ${token}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ title: 'Test todo', priority: 'high' }),
    })
    expect(res.statusCode).toBe(201)
    const todo = JSON.parse(res.body)
    expect(todo.title).toBe('Test todo')
    expect(todo.userId).toBe(userId)
  })

  it('POST /todos returns 401 without auth token', async () => {
```

```
    const res = await app.inject({
      method: 'POST',
      url: '/todos',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'Test' }),
    })
    expect(res.statusCode).toBe(401)
  })

  it('POST /todos returns 400 with empty title', async () => {
    const res = await app.inject({
      method: 'POST',
      url: '/todos',
      headers: {
        Authorization: `Bearer ${token}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ title: '' }),
    })
    expect(res.statusCode).toBe(400)
  })
})
```

7.4 E2E tests — test like a real user

```
// frontend/e2e/todos.spec.ts
import { test, expect } from '@playwright/test'

test.describe('Todo App', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/login')
    await page.fill('[name=email]', 'e2e@test.com')
    await page.fill('[name=password]', 'password123')
    await page.click('[type=submit]')
    await page.waitForURL('/dashboard')
  })

  test('can create a new todo', async ({ page }) => {
    await page.goto('/dashboard')
    await page.fill('[placeholder="What needs to be done?"]', 'Buy groceries')
    await page.click('[type=submit]')
    await expect(page.getByText('Buy groceries')).toBeVisible()
  })

  test('can mark todo as complete', async ({ page }) => {
    await page.goto('/dashboard')
    await page.click('[data-testid="todo-toggle"]')
    await expect(page.locator('[data-testid="todo-item"]').first())
      .toHaveClass(/completed/)
  })
})
```

```
// playwright.config.ts
import { defineConfig } from '@playwright/test'

export default defineConfig({
  testDir: './e2e',
  use: {
    baseURL: 'http://localhost:3000',
    screenshot: 'only-on-failure',
  },
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
})
```

7.5 Vitest config

```
// api/vitest.config.ts
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    environment: 'node',
    globals: true,
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
    },
  },
})
```

```
npm test              # Run once
npm run test:watch    # Watch mode
npm run test:coverage # Coverage report
```

7.6 Testing checklist

- □ CI fails if tests fail (enforced in GitHub Actions)
- □ Test DB isolated from dev (separate `DATABASE_URL` in CI)
- □ Unit tests for all Zod schemas
- □ Integration test for every route (at least happy path + 401)
- □ One E2E test for the most important user journey
- □ No flaky `sleep()` — use `waitFor` in E2E

PART 8 — DevOps & CI/CD

8.1 Why Docker?

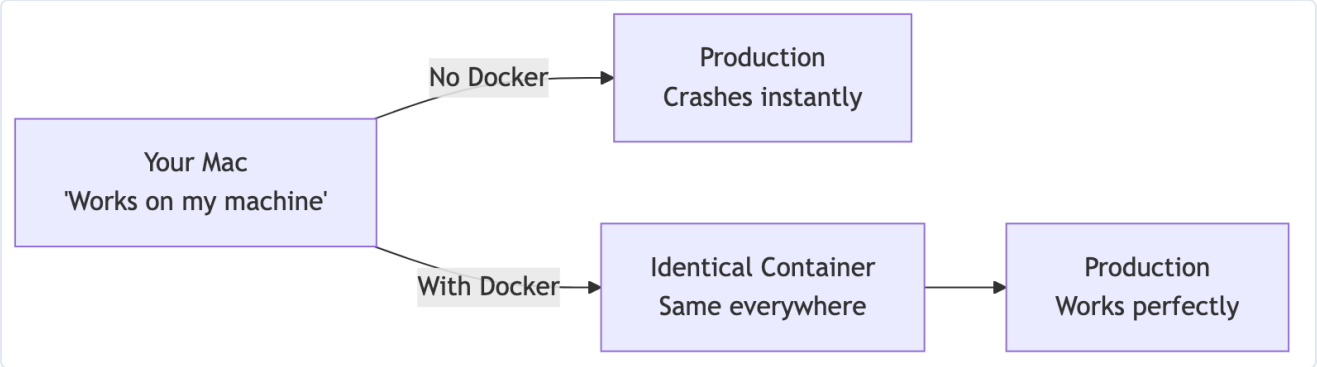


Figure 21 — 8.1 Why Docker?

8.2 Dockerfiles (multi-stage — the production pattern)

```
# api/Dockerfile
# Stage 1: Build
FROM node:22-alpine AS builder
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY prisma ./prisma
RUN npx prisma generate

COPY . .
RUN npm run build

# Stage 2: Production image (smaller — no build tools)
FROM node:22-alpine AS runner
WORKDIR /app

# Non-root user — security best practice
RUN addgroup -g 1001 -S nodejs && adduser -S nodeapp -u 1001

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./
COPY --from=builder /app/prisma ./prisma

USER nodeapp
EXPOSE 4000
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:4000/health || exit 1
CMD ["node", "dist/index.js"]
```

```
// frontend/next.config.js  ← required for the Dockerfile to work
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'standalone', // ← THIS LINE — without it, .next/standalone won't exist
                        //   and the production Docker image will crash at startup
  images: {
    remotePatterns: [
      // Add domains for any external images you load, e.g.:
      // { protocol: 'https', hostname: 'avatars.githubusercontent.com' },
    ],
  },
}

module.exports = nextConfig
```

> **Why `output: 'standalone'` ?** Next.js by default produces output that requires the full `node_modules` folder to run. With `standalone` , it bundles only what is actually used into a single `server.js` — making the Docker image 80–90% smaller and startup much faster.

```
# frontend/Dockerfile
FROM node:22-alpine AS builder
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app

RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

```
# .dockerignore (create this — critical for build speed and security)
node_modules
.git
.env
.env.local
.next
dist
coverage
*.log
```

8.3 GitHub Actions CI/CD pipeline (full)

```yaml
# .github/workflows/ci.yml
name: CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  test:
    name: Test & Lint
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: myapp_test
          POSTGRES_USER: myapp
          POSTGRES_PASSWORD: password
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:7-alpine
        ports: ['6379:6379']

    defaults:
      run:
        working-directory: api

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'
          cache-dependency-path: api/package-lock.json

      - name: Install dependencies
        run: npm ci

      - name: Run database migrations
        run: npx prisma migrate deploy
        env:
          DATABASE_URL: postgresql://myapp:password@localhost:5432/myapp_test

      - name: Run tests
        run: npm test
        env:
          DATABASE_URL: postgresql://myapp:password@localhost:5432/myapp_test
          REDIS_URL: redis://localhost:6379
          JWT_SECRET: test-secret-key-for-ci
          JWT_REFRESH_SECRET: test-refresh-secret-key-for-ci
          NODE_ENV: test

      - name: Build
        run: npm run build

  test-frontend:
    name: Build Frontend
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: frontend

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '22'
```

```
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json

      - run: npm ci
      - run: npm run build

  build-and-push:
    name: Build & Push Docker Images
    needs: [test, test-frontend]
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push API
        uses: docker/build-push-action@v5
        with:
          context: ./api
          push: true
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}-api:latest
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}-api:${{ github.sha }}

      - name: Build and push frontend
        uses: docker/build-push-action@v5
        with:
          context: ./frontend
          push: true
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}-frontend:latest
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}-frontend:${{ github.sha }}

  deploy-staging:
    name: Deploy to Staging
    needs: build-and-push
    runs-on: ubuntu-latest
    environment: staging

    steps:
      - name: Deploy to staging
        run: echo "Deploy ${{ github.sha }} to staging"
        # Replace with: railway up, fly deploy, aws ecs update-service, etc.

      - name: Smoke test
        run: |
          sleep 20
          curl -f https://staging.myapp.com/health || exit 1
```

8.4 Deployment strategies

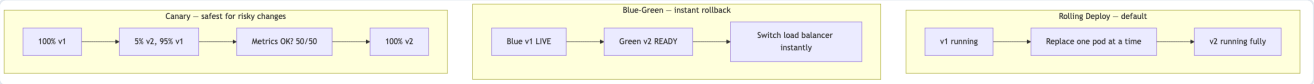


Figure 22 — 8.4 Deployment strategies

Strategy	Use when	Rollback speed
Rolling	Default for most apps	Minutes
Blue-Green	Need instant rollback, database migrations are risky	Seconds
Canary	High-risk changes, large user base	Minutes

8.5 CI/CD checklist

- ☐ `npm test` runs and fails CI if tests fail

- □ Docker images tagged with git SHA (not just `latest`)
- □ Images scanned for vulnerabilities (Trivy or Snyk)
- □ Staging deploys automatically on `main` merge
- □ Production deployment requires manual approval
- □ Rollback = redeploy previous image tag (documented)
- □ DB migrations run BEFORE app deploys

PART 9 — Cloud & Deploy

9.1 Easiest path — Railway or Fly.io (start here)

```
# Railway (simplest — web UI + CLI)
npm install -g @railway/cli
railway login
railway init
railway up

# Set env vars (never in code)
railway variables set DATABASE_URL=...
railway variables set JWT_SECRET=...

# Fly.io (great for global deploys)
brew install flyctl
flyctl auth login
flyctl launch
flyctl deploy
flyctl secrets set JWT_SECRET=...
```

9.2 AWS — production-grade setup

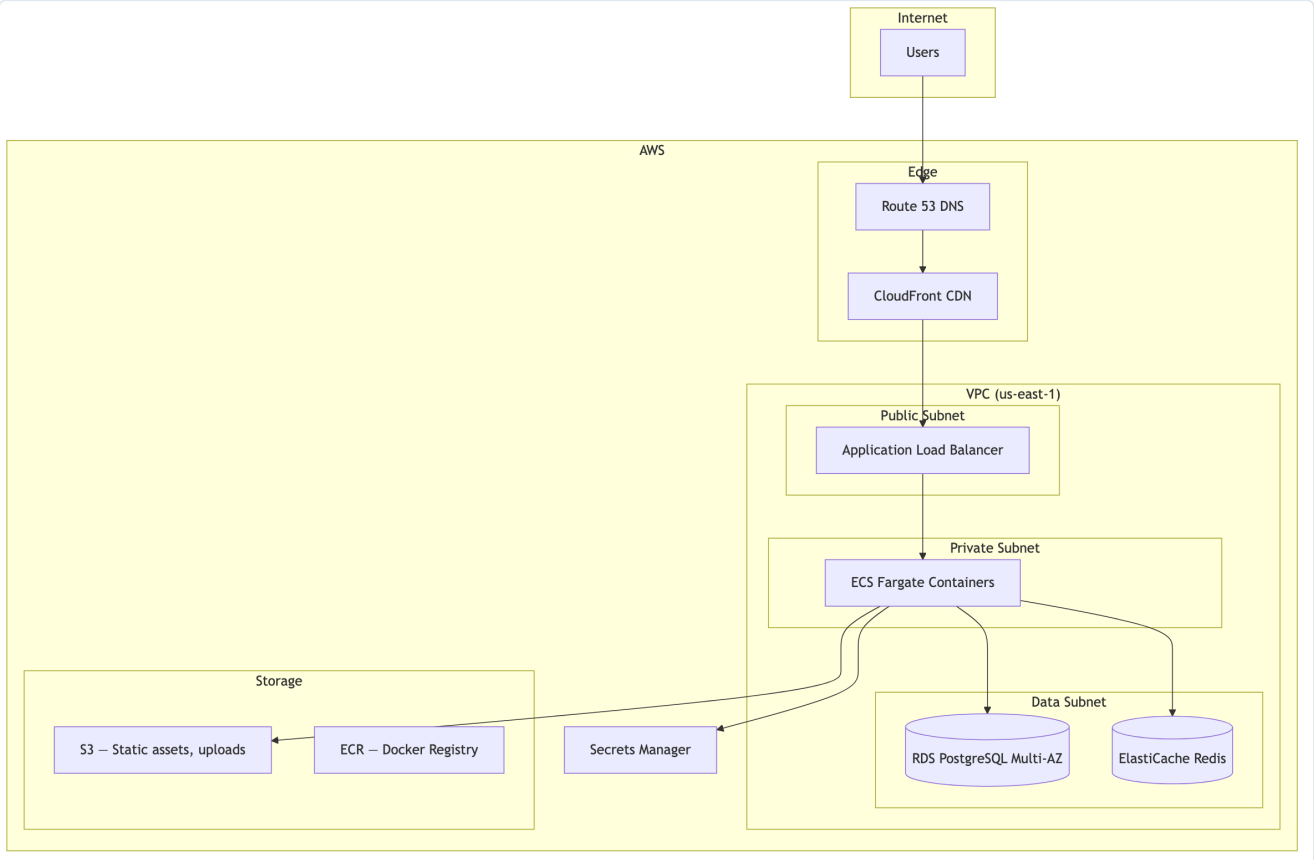


Figure 23 — 9.2 AWS — production-grade setup

9.3 Terraform (Infrastructure as Code)

```
# infrastructure/main.tf
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }

  backend "s3" {
    bucket = "my-app-tf-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}

provider "aws" {
  region = "us-east-1"
}

# VPC with public/private subnets
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.0"

  name            = "my-app-vpc"
  cidr            = "10.0.0.0/16"
  azs             = ["us-east-1a", "us-east-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]
  enable_nat_gateway = true
}

# RDS PostgreSQL
resource "aws_db_instance" "main" {
  identifier        = "myapp-postgres"
  engine            = "postgres"
  engine_version    = "16.1"
  instance_class    = "db.t3.medium"
  allocated_storage = 20
  storage_encrypted = true

  db_name  = "myapp"
  username = "myapp"
  password = var.db_password  # from Secrets Manager

  multi_az               = true  # high availability
  backup_retention_period = 7
  skip_final_snapshot     = false

  vpc_security_group_ids = [aws_security_group.rds.id]
  db_subnet_group_name   = aws_db_subnet_group.main.name
}
```

```
terraform init    # Download providers
terraform plan    # Preview what will change
terraform apply   # Apply changes (confirm yes)
# Never run terraform destroy on production without a backup
```

9.4 Environments

Env	Purpose	Data	Who deploys
local	Your laptop	Fake / seed	You
staging	Pre-prod testing	Anonymized copy	CI automatically
production	Real users	Real data	Manual approval

9.5 Deploy checklist

- □ Secrets set in hosting platform, not in `.env` file in repo
- □ Custom domain with HTTPS certificate
- □ DB automated backups enabled
- □ Health check endpoint live: `GET /health`

- □ Rollback plan documented (which image tag to redeploy)
- □ Staging smoke test passing before promoting to production

---

PART 10 — Observability & Monitoring

10.1 The three pillars

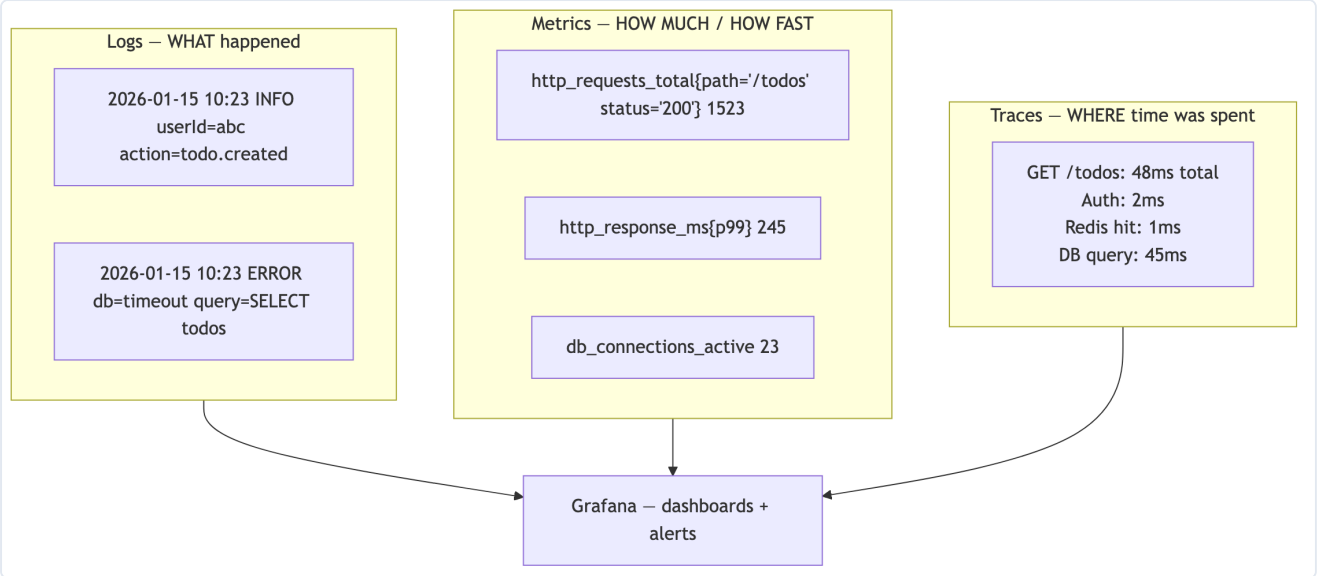


Figure 24 — 10.1 The three pillars

10.2 Structured logging (always JSON)

```
// api/src/lib/logger.ts
import pino from 'pino'

export const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  transport: process.env.NODE_ENV === 'development'
    ? { target: 'pino-pretty' } // human-readable in dev
    : undefined,                // raw JSON in production
  base: {
    service: 'my-backend',
    version: process.env.APP_VERSION || '1.0.0',
    env: process.env.NODE_ENV,
  },
})

// ✅ Good log — has context
logger.info({ userId: '123', action: 'todo.created', todoId: 'abc' }, 'Todo created')
logger.error({ err, userId: '123', url: '/todos' }, 'Failed to fetch todos')

// ❌ Bad log — no context, can't debug from this
console.log('Todo created')
```

10.3 Request logging in Fastify

```
// Fastify has pino built in — every request is automatically logged:
// { "reqId": "1", "req": { "method": "POST", "url": "/todos" }, ... }

// Add request IDs to every log via hooks:
server.addHook('onRequest', (request, reply, done) => {
  request.log.info({ method: request.method, url: request.url }, 'Incoming request')
  done()
})

server.addHook('onResponse', (request, reply, done) => {
  request.log.info(
    { method: request.method, url: request.url, statusCode: reply.statusCode, duration: reply.elapsedTime },
    'Request completed'
  )
  done()
})
```

10.4 OpenTelemetry — distributed tracing

```
// api/src/lib/telemetry.ts — import this FIRST in index.ts
import { NodeSDK } from '@opentelemetry/sdk-node'
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http'
import { Resource } from '@opentelemetry/resources'
import { SemanticResourceAttributes } from '@opentelemetry/semantic-conventions'
import { HttpInstrumentation } from '@opentelemetry/instrumentation-http'

const sdk = new NodeSDK({
  resource: new Resource({
    [SemanticResourceAttributes.SERVICE_NAME]: 'my-backend',
    [SemanticResourceAttributes.SERVICE_VERSION]: process.env.APP_VERSION || '1.0.0',
  }),
  traceExporter: new OTLPTraceExporter({
    url: process.env.OTEL_ENDPOINT || 'http://localhost:4318/v1/traces',
  }),
  instrumentations: [new HttpInstrumentation()],
})

sdk.start()
process.on('SIGTERM', () => sdk.shutdown())
```

10.5 Alerts — get notified before users complain

Alert	Threshold	Action
5xx rate	> 1% of requests	Page on-call immediately
Health check down	2 consecutive fails	Page on-call immediately
p99 latency	> 2 seconds	Investigate within 1 hour
DB connections	> 80% of pool	Scale DB or add connection pooler
Disk usage	> 85%	Add storage or rotate logs

10.6 Minimum viable observability checklist

- □ JSON logs to stdout (hosting platform aggregates them)
- □ `GET /health` returns DB connectivity status
- □ Dashboard: RPS, error rate, p95 latency
- □ Alert on: health check down, 5xx spike
- □ At least one alerting channel (Slack webhook or email)

PART 11 — Performance

11.1 Find slowness before optimizing

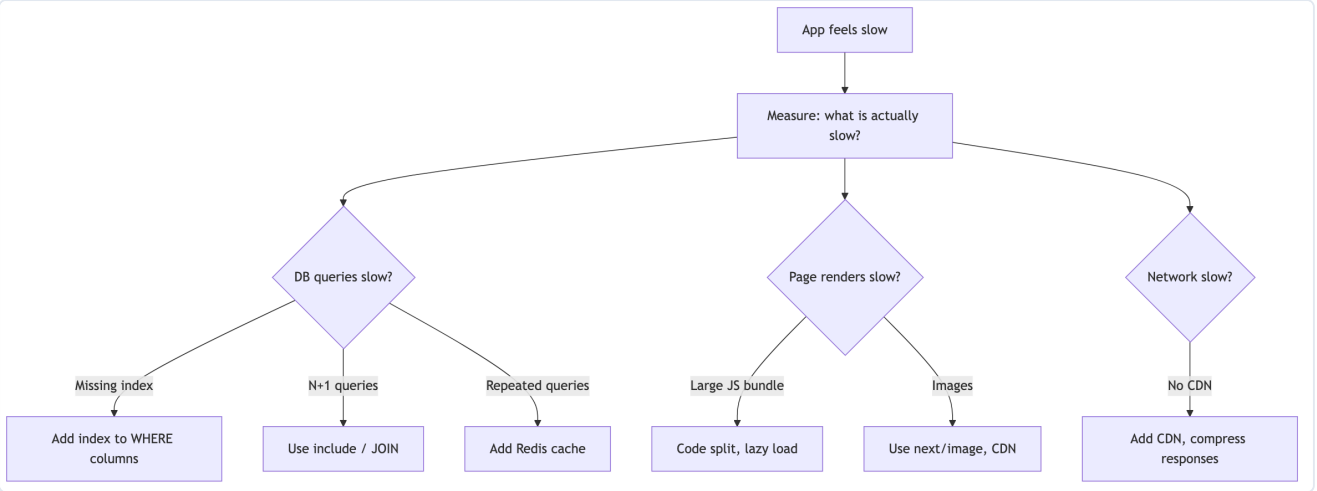


Figure 25 — 11.1 Find slowness before optimizing

> **Rule:** Measure first. Never add caching without proving the query is slow.

11.2 Core Web Vitals

Metric	Target	Fix
LCP (Largest Contentful Paint)	< 2.5s	Preload hero image, use CDN, optimize images
INP (Interaction to Next Paint)	< 200ms	Code split, reduce JS bundle
CLS (Cumulative Layout Shift)	< 0.1	Set width/height on images and iframes

```
// ✅ Always use Next.js Image (auto-optimized)
import Image from 'next/image'
<Image src="/hero.jpg" alt="Hero" width={1200} height={600} priority />

// ✅ Lazy load heavy components
import dynamic from 'next/dynamic'
const HeavyChart = dynamic(() => import('@/components/chart'), {
  loading: () => <p>Loading...</p>,
  ssr: false,
})
```

11.3 Caching layers

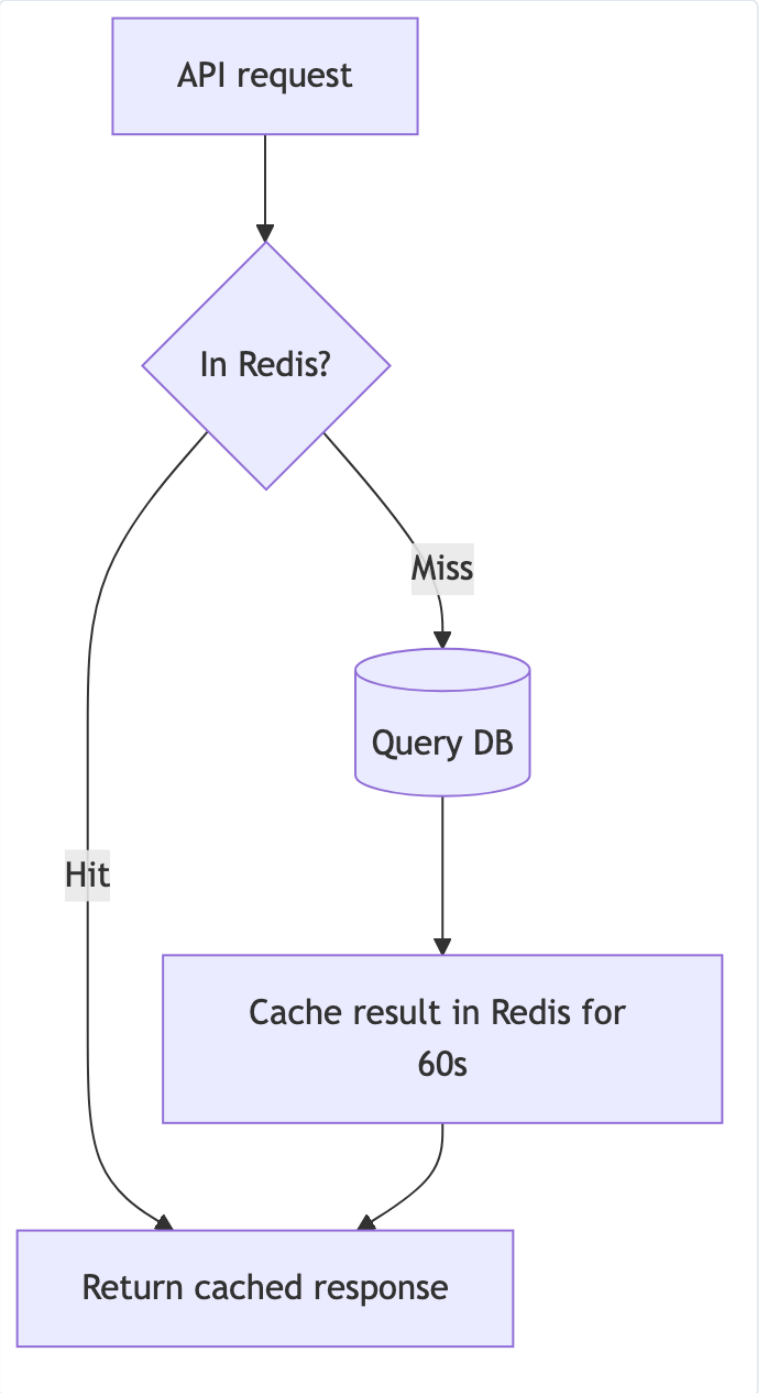


Figure 26 — 11.3 Caching layers

11.4 Scaling ladder

Users (concurrent)	Action
< 500	Optimize queries, add missing indexes
500–2k	Cache hot reads in Redis
2k–10k	Read replica, more API replicas + load balancer
10k+	Split domains, queue heavy work

11.5 Performance checklist

- ☐ `EXPLAIN ANALYZE` checked for queries on tables > 10k rows
- ☐ Indexes on all `WHERE` columns (foreign keys, email, status)

- □ No `SELECT *` on hot paths — select specific columns
- □ Images: next/image or CDN, not raw `<img>`
- □ JS bundle: check `npm run build` output, no accidental huge imports
- □ Redis cache for repeated, expensive reads

PART 12 — Production Launch Checklist

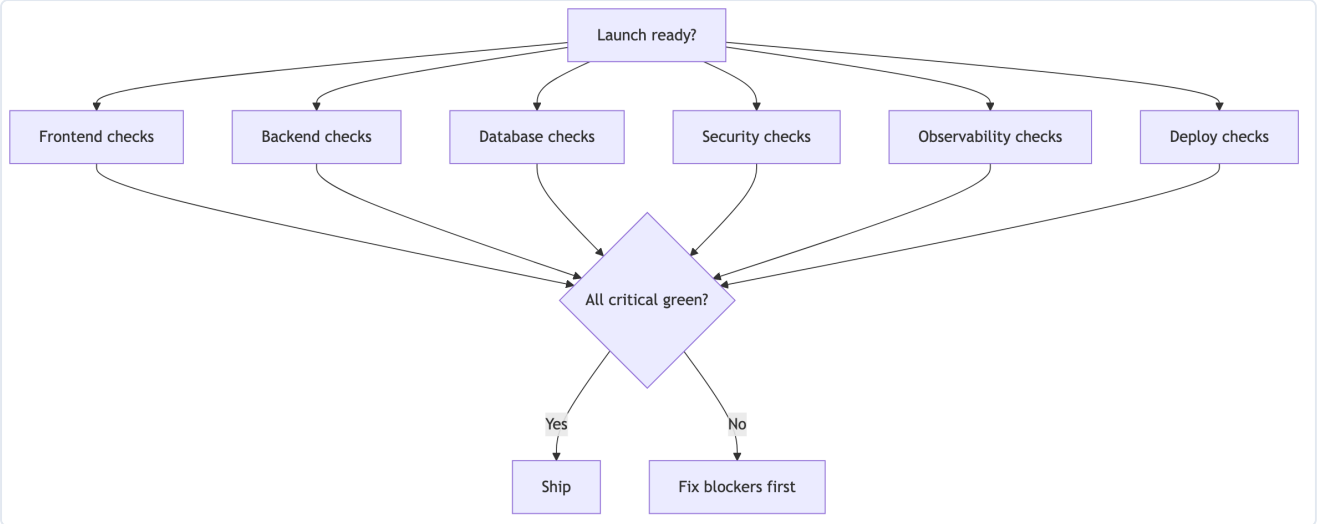


Figure 27 — 11.5 Performance checklist

Critical (must be green before shipping)

Frontend

- ☐ Loading, error, and empty states on every data view
- ☐ Env vars set in hosting platform (not hardcoded)
- ☐ No API keys or secrets in client bundle
- ☐ Forms: disabled during submit, validation errors shown

Backend

- ☐ Auth enforced on all private routes
- ☐ Input validation on all write endpoints
- ☐ `GET /health` endpoint returns 200
- ☐ Global error handler — no stack traces to clients
- ☐ Graceful shutdown on SIGTERM

Database

- ☐ Migrations applied successfully
- ☐ Automated backups enabled
- ☐ Indexes on columns used in `WHERE` and `JOIN`
- ☐ Connection pool size set appropriately

Security

- ☐ HTTPS with valid certificate
- ☐ Secrets in env/vault, not in code or git
- ☐ Rate limiting on auth endpoints
- ☐ CORS allowing only your frontend origin
- ☐ Helmet security headers

Observability

- ☐ Logs visible in log aggregator

- □ Alert on 5xx spike and health check failure
- □ On-call knows how to rollback

Deploy

- □ Rollback procedure documented (previous image tag)
- □ Staging smoke test: health + login + create item
- □ DB migration ran BEFORE app deployed

Launch readiness score

Domain	Score 1–5	Blocker if < 3?
Frontend UX		No
Backend API		Yes
Database		Yes
Security		Yes
Observability		Yes
Deploy safety		Yes

Ship when: all "Yes" domains score 4+, nothing at 1–2 without mitigation.

PART 13 — Troubleshooting (25 fixes)

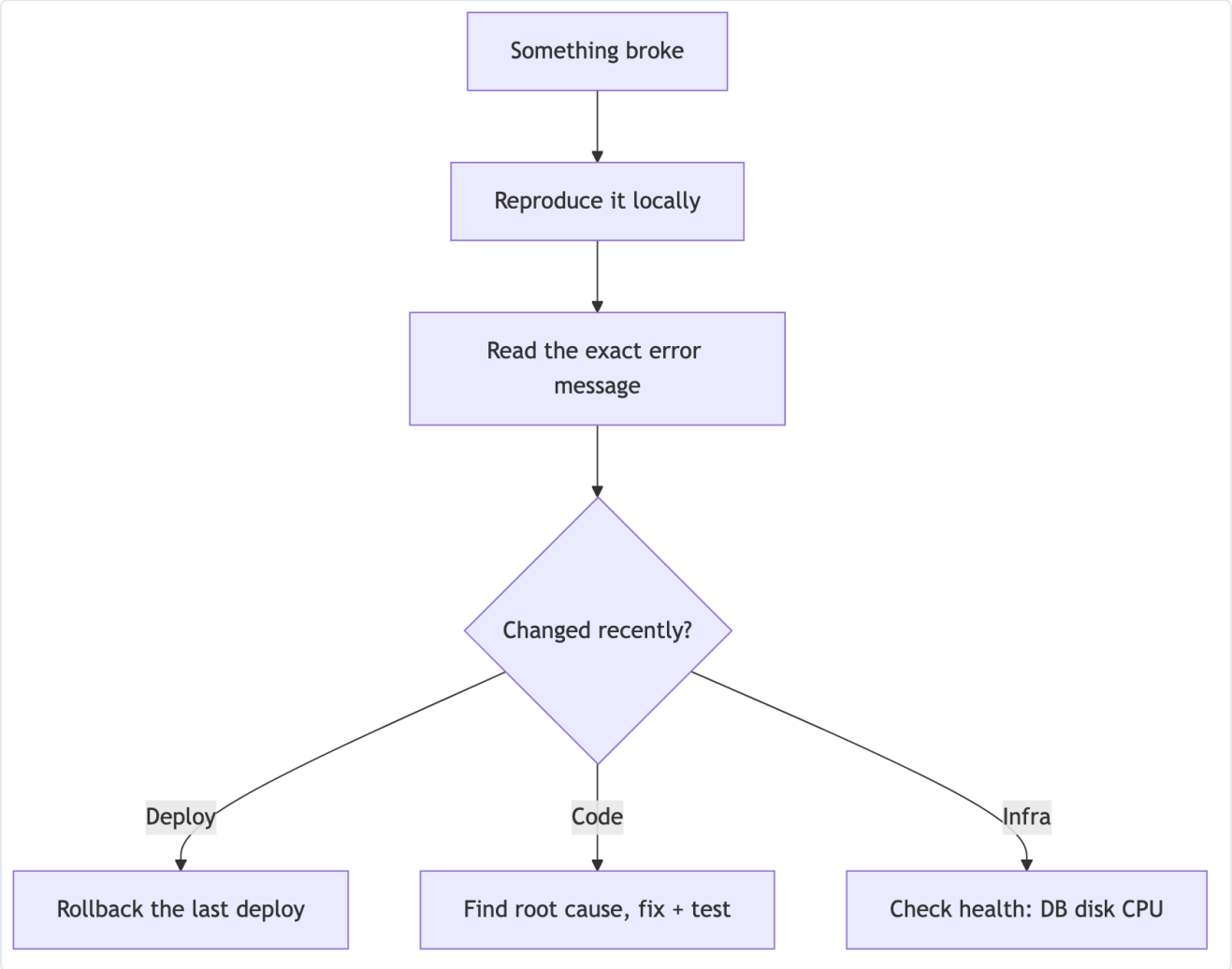


Figure 28 — Launch readiness score

Problem	Likely cause	Fix
`Cannot find module 'fastify-cors'`	Wrong package name	Change to `@fastify/cors` — the old package is abandoned
API not responding at all	CORS blocked	Register `@fastify/cors` with your frontend origin
Frontend can't reach API	Wrong URL in env	Set `NEXT_PUBLIC_API_URL=http://localhost:4000`
`Error: listen EADDRINUSE :3000`	Port already in use	`lsof -ti:3000 | xargs kill -9`
DB connection fails	Postgres not running	`docker-compose up postgres`
`Error: .env file missing`	No env file	`cp .env.example .env` then fill in values
Docker build fails slowly	No `.dockerignore`	Add `.dockerignore` excluding `node_modules`
`401 Unauthorized` on every request	JWT secret wrong	Check `JWT_SECRET` matches between signing and verifying
`401 Unauthorized` after 15 minutes	Access token expired	Implement refresh token flow
Migration failed in production	Wrong order	Restore backup, fix migration, retry
API slow on list endpoint	No index	`CREATE INDEX idx_todos_user_id ON todos(user_id)`
No logs in production	Wrong log format	Log JSON to stdout — platform collects stdout
Docker image is 1GB+	Single-stage Dockerfile	Use multi-stage build — only copy `dist/`
Tests pass locally, fail in CI	Different env vars	Add `.env.test` + set CI env variables
Secret committed to git	Panic	Rotate secret immediately, add to `.gitignore`, use `git filter-repo` to purge
Random 502 Bad Gateway	App crashing on unhandled error	Add global error handler, fix the bug
Stale deploy (new code not live)	Cached Docker image	Tag with `$(git rev-parse --short HEAD)`
HTTPS cert error	Cert expired	Renew cert in load balancer / cert-manager
Disk full in production	Unbounded logs	Set log rotation policy + retention
Flaky E2E tests	Race condition	Use `await expect(locator).toBeVisible()` not sleep
CORS preflight failing	Missing OPTIONS method	Add `methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE']` to CORS config
Postgres connection pool exhausted	Too many connections	Set `connection_limit` in DATABASE_URL or use PgBouncer

Problem	Likely cause	Fix
`PrismaClientKnownRequestError P2025`	Record not found	Add null check before using result
`TypeError: Cannot read property of undefined`	Missing loading state	Add `if (isLoading) return` in your component
E2E test can't log in	Test user not seeded	Add seed script or create user in `beforeAll`

PART 14 — Real-World App Blueprints

Blueprint A — SaaS MVP (the standard fresher project)

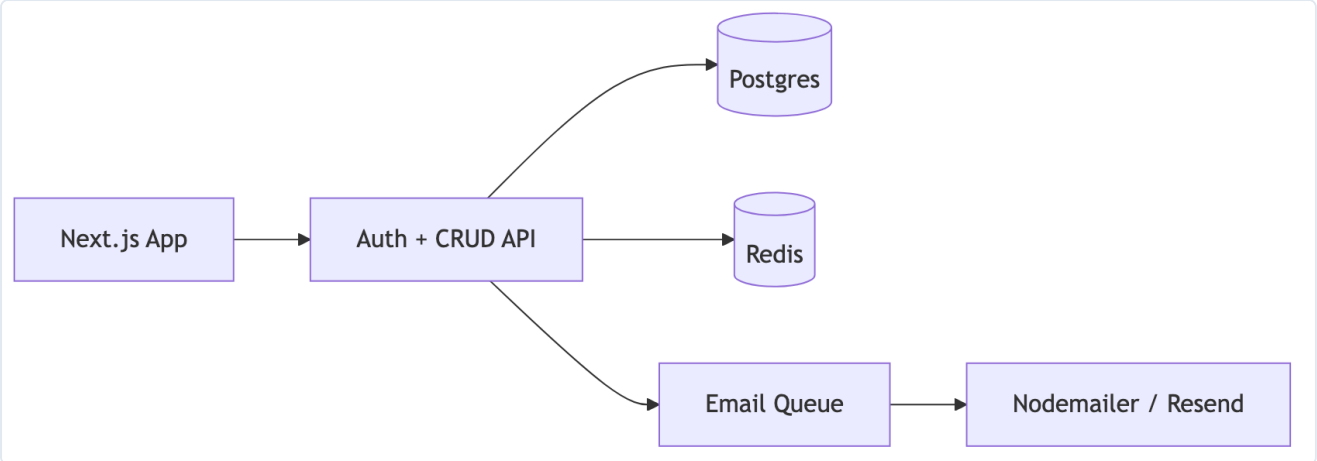


Figure 29 — Blueprint A — SaaS MVP (the standard fresher project)

Week	What to build
1	Auth + organization + user model
2	Core entity CRUD (the thing users do)
3	Email notifications + billing (Stripe)
4	Admin dashboard + analytics

Blueprint B — E-commerce (simplified)

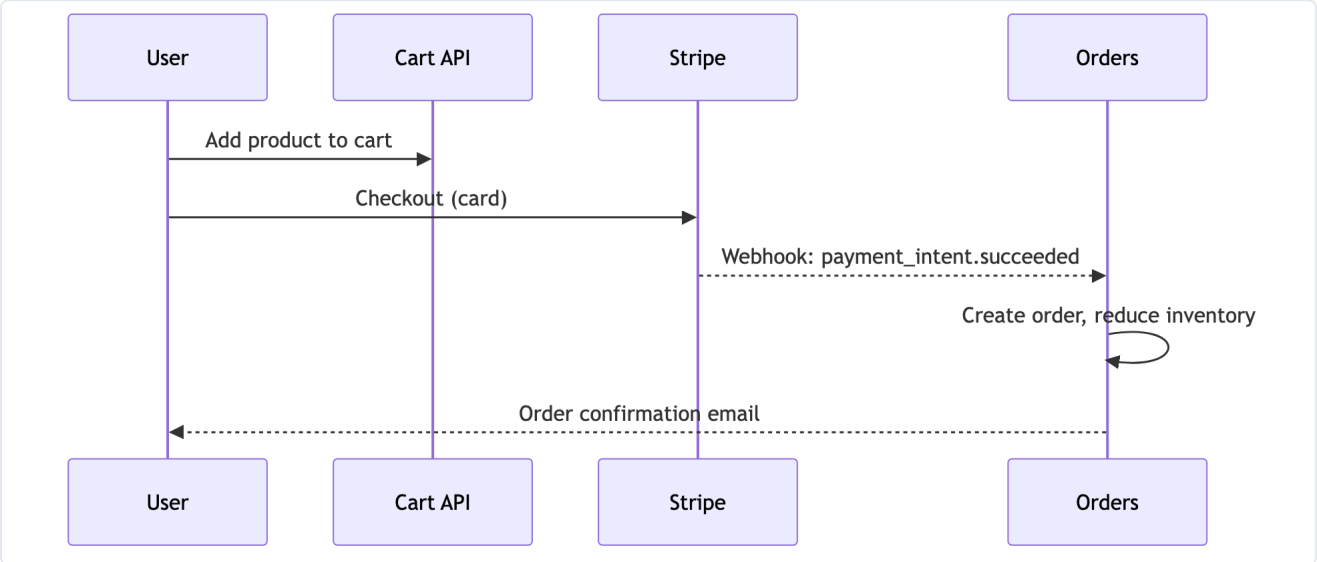


Figure 30 — Blueprint B — E-commerce (simplified)

Key tables: `users` , `products` , `carts` , `cart_items` , `orders` , `order_items` , `payments`

Blueprint C — Realtime chat

- WebSocket server via `ws` package or managed (Pusher, Ably)
- Redis pub/sub for distributing messages across API instances
- Postgres for message history, Redis for recent messages cache

Blueprint D — Internal admin tool

```
Week 1: Next.js + Fastify + Postgres — list views and edit forms
Week 2: Auth with SSO (NextAuth + Google/GitHub)
Week 3: Data export (CSV), audit log
Week 4: Role-based permissions (ADMIN vs USER)
```

System design thinking (for interviews + real work)

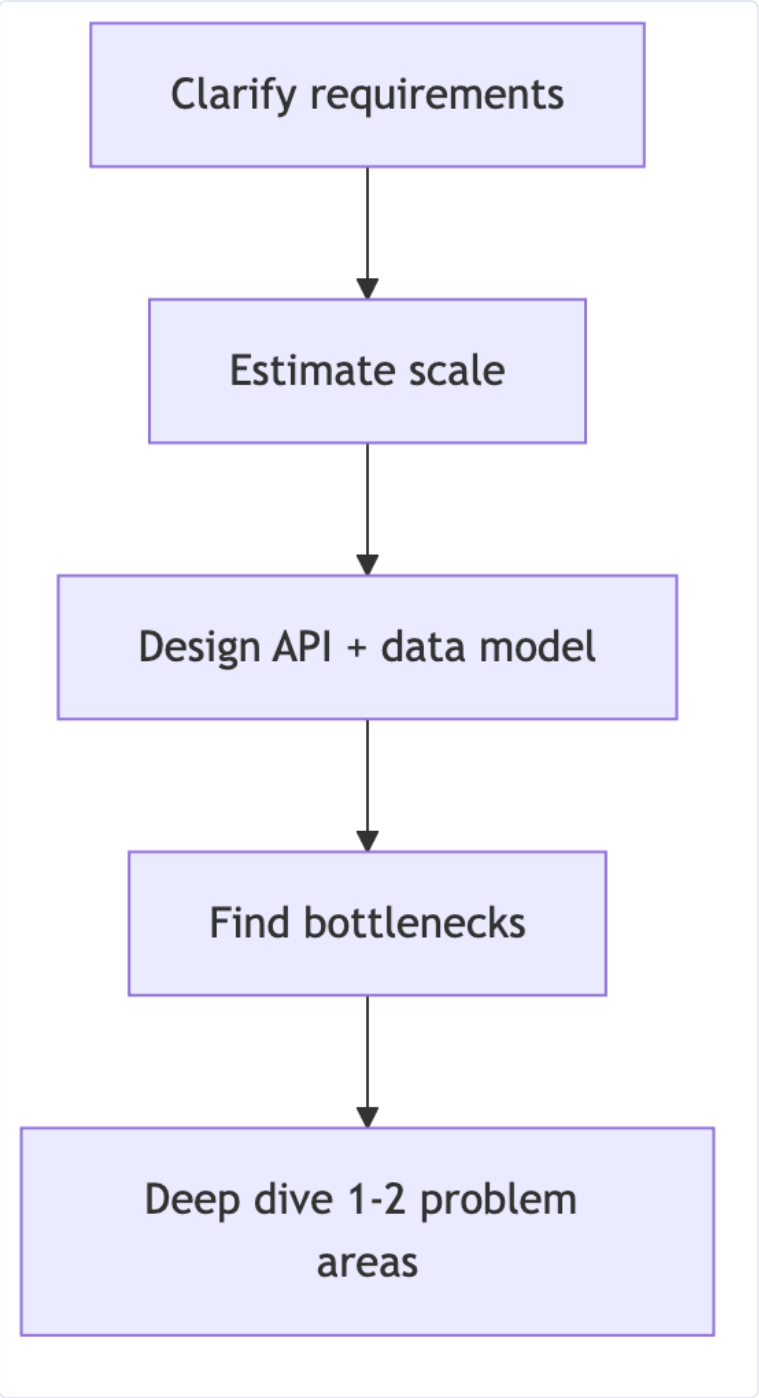


Figure 31 — System design thinking (for interviews + real work)

Step	Questions to ask
Clarify	Who uses it? Read-heavy or write-heavy? Consistency needed?
Estimate	100 users vs 1M users changes the design
Design	Boxes: client, API, DB, cache, queue
Bottlenecks	Single points of failure? Slow queries? N+1?

PART 15 — Team & Git Workflow

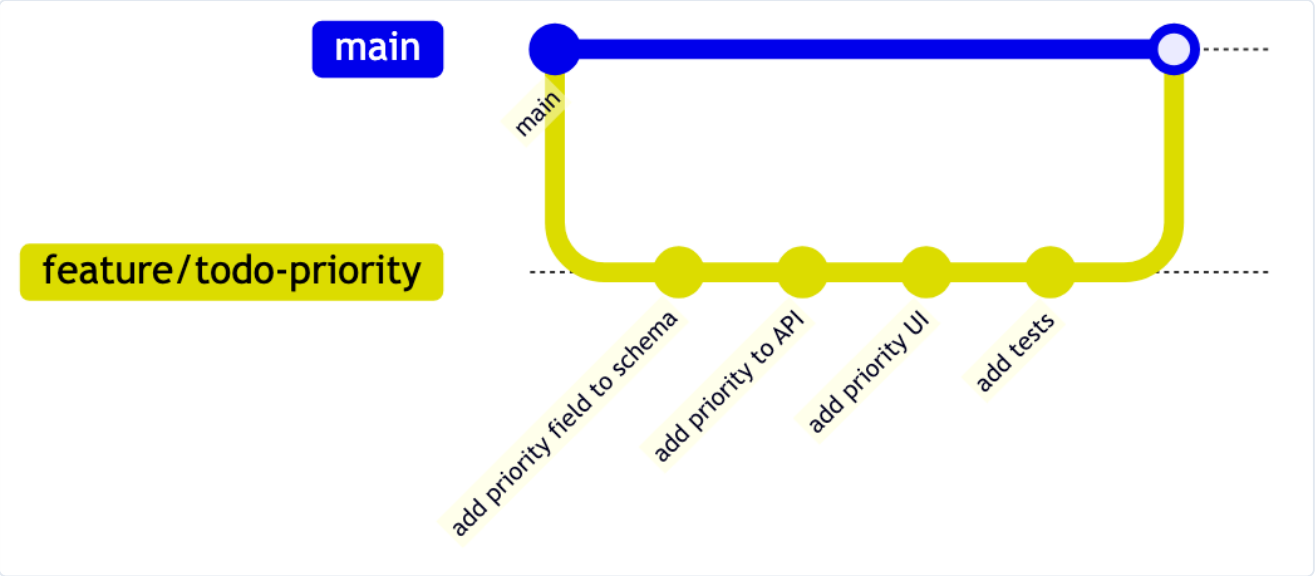


Figure 32 — System design thinking (for interviews + real work)

Branch and PR rules

Practice	Rule
Branch naming	`feature/short-description` , `fix/bug-name` , `chore/dependency-update`
PR size	< 400 lines ideal — smaller = faster reviews
Merging	1 approval + CI green = merge
Commits	`feat:` , `fix:` , `chore:` , `docs:` prefixes + why in body

Commit message format

```
feat: add priority field to todos

Users need to distinguish urgent tasks. Added 'priority' enum
(low, medium, high) to todos table, API, and UI filter.

Closes #42
```

PR checklist

- □ Tests added or updated for the change
- □ No secrets committed (double-check `.env`)
- □ Screenshots/GIF for UI changes
- □ Migration noted in description if DB schema changed
- □ Breaking changes documented

Code review rules

Role	Responsibility
Author	Small PRs, clear description, tests included
Reviewer	Review within 24h, suggest not demand, approve when logic is sound

PART 16 — Think Like a Senior Engineer

The 5 rules

#	Rule	Junior vs Senior
1	**Ship small**	Junior: 3-week branch. Senior: working PR daily
2	**Measure before optimizing**	Junior: adds Redis "for scale". Senior: runs `EXPLAIN ANALYZE` first
3	**Automate everything repetitive**	Junior: deploys by hand. Senior: CI/CD, zero manual steps
4	**Simple > clever**	Junior: fancy abstractions. Senior: readable code that works
5	**Secure by default**	Junior: "we'll add auth later". Senior: auth is Day 4

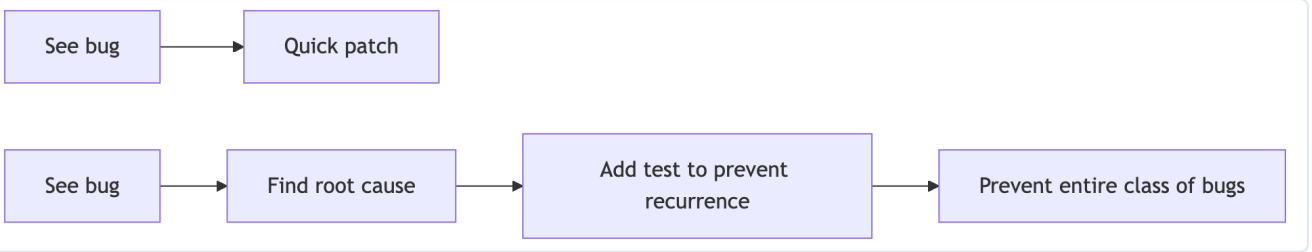


Figure 33 — The 5 rules

Architecture Decision Records (ADR)

When you make a significant technical decision, write it down. One paragraph each:

```
# ADR 001: Use JWT + refresh tokens for auth

## Status: Accepted

## Decision
Use JWT access tokens (15min) + refresh tokens stored in sessions table.

## Why
- Access tokens are stateless — no DB lookup on every request
- Refresh tokens in DB allow logout/revocation
- 15min access token limits damage if stolen

## Trade-offs
- Slightly more complex than sessions
- Refresh token rotation required
- DB lookup only on token refresh (acceptable)
```

PART 17 — 90-Day Growth Path

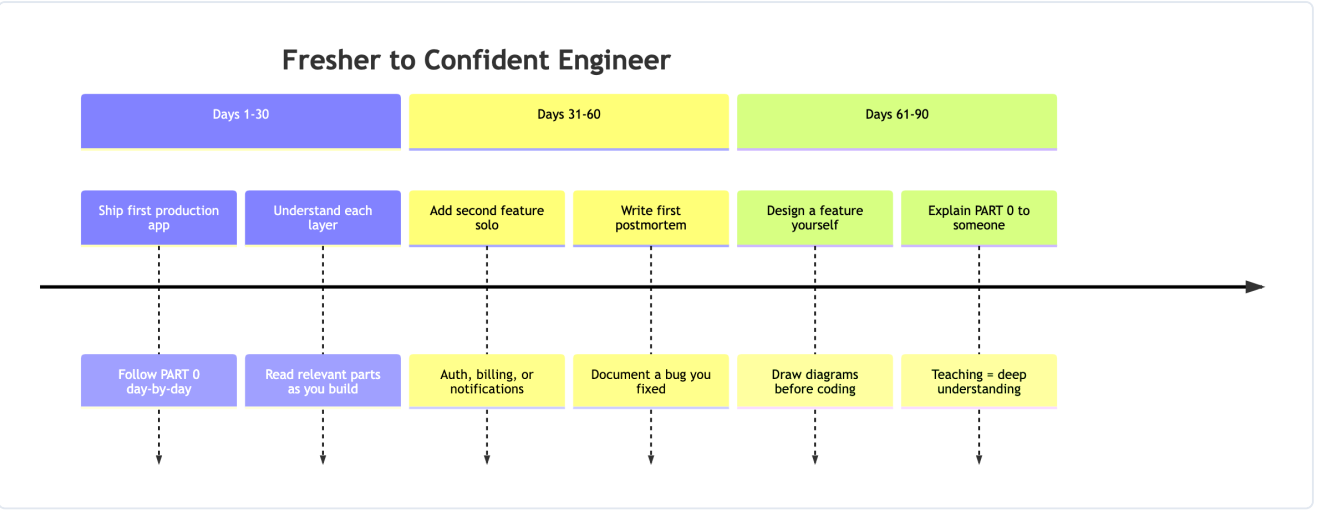


Figure 34 — Architecture Decision Records (ADR)

Month	Focus	Proof
1	First app live with real URL	Deployed URL + monitoring dashboard
2	Code quality	Tests added, fewer regressions
3	Design thinking	Architecture diagram for next feature
4	Advanced topics	OAuth, Prisma, queue, cloud deploy
5	Observability	Prometheus + Grafana dashboard running
6	System design	Second service: payments, notifications

Learning advice: Deployed beats perfect. One working app beats ten half-started ones.

Appendix A — Copy-Paste Commands

```
# === BOOTSTRAP A NEW PROJECT ===

# Frontend
npx create-next-app@latest frontend \
  --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
cd frontend
npm install @tanstack/react-query zustand react-hook-form zod @hookform/resolvers
npx shadcn-ui@latest init && npx shadcn-ui@latest add button input card

# Backend
mkdir api && cd api && npm init -y
npm install fastify @fastify/cors @fastify/helmet
npm install jsonwebtoken   # use jsonwebtoken directly — NOT @fastify/jwt
npm install @prisma/client zod bcrypt jsonwebtoken ioredis pino pino-pretty
npm install -D typescript tsx @types/node @types/bcrypt @types/jsonwebtoken prisma vitest

# Init prisma
npx prisma init
npx prisma migrate dev --name init

# === DOCKER ===
docker-compose up --build             # Start everything
docker-compose down                   # Stop
docker-compose down -v                # Stop + wipe data
docker build -t my-api:$(git rev-parse --short HEAD) ./api
docker push ghcr.io/YOUR_ORG/my-api:$(git rev-parse --short HEAD)

# === DATABASE ===
npx prisma migrate dev --name add-feature    # New migration
npx prisma migrate deploy                     # Apply in production
npx prisma studio                             # Open data browser
npx prisma db seed                            # Run seed script

# === TESTING ===
cd api && npm test                    # Run tests once
npm run test:watch                    # Watch mode
npm run test:coverage                 # Coverage report
npx playwright test                   # E2E tests

# === GENERATE SECRETS ===
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# === PORTS ===
lsof -ti:3000 | xargs kill -9         # Kill process on port 3000
lsof -ti:4000 | xargs kill -9         # Kill process on port 4000

# === HEALTH CHECK ===
curl -sf http://localhost:4000/health && echo "API OK"

# === AUTH FLOW (curl examples) ===
# Register
curl -s -X POST http://localhost:4000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"you@test.com","password":"secret123","name":"Test User"}' | jq .

# Login
TOKEN=$(curl -s -X POST http://localhost:4000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"you@test.com","password":"secret123"}' | jq -r '.accessToken')

# Access protected route
curl -s http://localhost:4000/auth/me \
  -H "Authorization: Bearer $TOKEN" | jq .
```

Appendix B — Environment Variables Reference

```
# === API (.env) ===
DATABASE_URL="postgresql://user:pass@localhost:5432/myapp"
REDIS_URL="redis://localhost:6379"
JWT_SECRET="generate-with-crypto-random-32-bytes"
JWT_REFRESH_SECRET="different-generate-with-crypto-random-32-bytes"
PORT=4000
NODE_ENV=development                    # development | production | test
ALLOWED_ORIGINS="http://localhost:3000"   # comma-separated
LOG_LEVEL=info                          # debug | info | warn | error
APP_VERSION=1.0.0
OTEL_ENDPOINT=http://localhost:4318/v1/traces   # optional, for tracing

# === FRONTEND (.env.local) ===
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_APP_URL=http://localhost:3000

# === CI / Production extras ===
DATABASE_URL="postgresql://user:pass@rds-host:5432/myapp"
REDIS_URL="redis://elasticache-host:6379"
ALLOWED_ORIGINS="https://myapp.com"
NODE_ENV=production
```

Rule	Details
Never commit `.env`	Add to `.gitignore` on day 1
Always commit `.env.example`	Empty values as documentation
`NEXT_PUBLIC_` is public	Only use for URLs, never secrets
Rotate `JWT_SECRET` periodically	Rotating invalidates all tokens — warn users
Use secret manager in prod	AWS Secrets Manager, Vault, Railway secrets

Appendix C — Error Decoder

Error message	What it means	How to fix
`Cannot find module '@fastify/cors'`	Package not installed	`npm install @fastify/cors`
`Cannot find module 'fastify-cors'`	Old abandoned package	Change to `@fastify/cors` in package.json and import
`EADDRINUSE 0.0.0.0:4000`	Port 4000 already in use	`lsof -ti:4000 | xargs kill -9`
`ECONNREFUSED 127.0.0.1:5432`	Postgres not running	`docker-compose up postgres`
`P2002: Unique constraint failed`	Duplicate email/value	Return 409 with `Email already in use`
`P2025: Record not found`	Prisma can't find the row	Add null check: `if (!result) return 404`
`JsonWebTokenError: invalid signature`	Wrong `JWT_SECRET`	Verify same secret used for sign and verify
`TokenExpiredError: jwt expired`	Token is too old	Return 401 with `TOKEN_EXPIRED` — client should refresh
`invalid input syntax for type uuid`	Passing non-UUID as ID	Validate ID format with Zod before DB query
`CORS error: No 'Access-Control-Allow-Origin'`	CORS not set up	Register `@fastify/cors` with frontend origin
`Hydration mismatch`	Server and client render different HTML	Add `suppressHydrationWarning` or move to Client Component
`useQuery is not a function`	Missing QueryClientProvider	Wrap your app in `<QueryClientProvider>`
`prisma.todo.findMany is not a function`	Prisma client not generated	Run `npx prisma generate`
`Can't reach database server at postgres:5432`	Wrong DB host in Docker	Use service name `postgres` not `localhost` in docker-compose
`TypeError: Cannot read properties of undefined (reading 'map')`	Data is undefined before fetch	Add `const { data = [] }` or loading check
`npm ERR! peer dep missing`	Incompatible package versions	Check peer deps, use `--legacy-peer-deps` as last resort

Appendix D — Glossary

Term	Meaning (30 seconds)
API	Backend HTTP service that your UI calls
CRUD	Create, Read, Update, Delete — the four database operations
JWT	Signed token proving who the user is without hitting DB every time
bcrypt	Algorithm for hashing passwords so you never store plain text
ORM	Library (Prisma) that lets you query DB with TypeScript instead of raw SQL
Migration	A versioned change to your DB schema, run in order
Docker	Packages your app + all dependencies into a container that runs identically everywhere
CI/CD	Automated pipeline: test → build → deploy on every push
CORS	Browser security that blocks frontend from calling a different-origin API unless allowed
Rate limiting	Block users who send too many requests (prevents brute force and abuse)
N+1 query	Bug: fetching 10 users then 10 separate queries for their todos = 11 queries. Use JOIN instead
Index	DB data structure that makes lookups fast. Add to columns you filter on
p95 latency	95% of requests are faster than this. p99 = 99th percentile
Rollback	Deploy the previous known-good version of your app
Staging	Production-like environment for testing before real users see changes
TTL	Time-To-Live — how long a cache entry stays before expiring
Idempotent	Same request twice = same result (safe to retry)
Load balancer	Distributes incoming requests across multiple API instances
Connection pool	Reuses DB connections instead of opening one per request
Structured logs	JSON logs with fields (userId, action, level) instead of plain text strings
Graceful shutdown	Finish in-flight requests before stopping the server on SIGTERM
Health check	`GET /health` endpoint that returns 200 when app is ready

Term	Meaning (30 seconds)
ADR	Architecture Decision Record — a short doc explaining why you made a technical choice

You are ready

You now have everything to ship a production-grade app from scratch:

- **PART 0** — 10-day checklist from zero to live
- **PARTS 1-5** — Complete setup, architecture, frontend, backend, database code
- **PART 6** — Full auth with JWT + refresh tokens
- **PARTS 7-10** — Tests, Docker, CI/CD, cloud deploy, monitoring
- **PARTS 11-12** — Performance tuning + launch checklist
- **PART 13** — 25 troubleshooting fixes with exact commands
- **PARTS 14-17** — Blueprints, Git workflow, senior mindset, growth path
- **Appendices** — Commands, env vars, error decoder, glossary

Next steps right now

1. Open `starter-app/` and run `docker-compose up --build`
2. Follow PART 0 day by day — check every box
3. Deploy to staging on Day 8
4. Run PART 12 launch checklist before shipping
5. Build your second feature without re-reading everything — use the table of contents at the top

Every senior engineer shipped a broken toy app first. Yours is next. Ship small. Measure. Repeat.

Version 3.0 — The Complete Edition. Stack: Next.js · Fastify · PostgreSQL · Prisma · Redis · Docker · GitHub Actions.

Index

Click any term to jump to that section in the book.

Term	Jump to
10-day build plan	Go to section →
Authentication (JWT + bcrypt)	Go to section →
AWS ECS + Terraform	Go to section →
bcrypt	Go to section →

Term	Jump to
BullMQ (background jobs)	Go to section →
CI/CD pipeline	Go to section →
CRUD routes	Go to section →
Docker multi-stage	Go to section →
Error decoder	Go to section →
E2E tests (Playwright)	Go to section →
Fastify	Go to section →
GitHub Actions	Go to section →
Glossary	Go to section →
JWT refresh tokens	Go to section →
Launch checklist	Go to section →
Microservices	Go to section →
Migrations (Prisma)	Go to section →
N+1 query problem	Go to section →
Next.js 15 (App Router)	Go to section →
Node.js	Go to section →
NoSQL vs SQL	Go to section →
Observability	Go to section →
OpenTelemetry (tracing)	Go to section →
Pino (structured logging)	Go to section →
PostgreSQL	Go to section →
Prisma ORM	Go to section →
React Query (TanStack v5)	Go to section →
Redis (caching)	Go to section →
Repository pattern	Go to section →
Security (OWASP, CORS, CSP)	Go to section →
Server Components (RSC)	Go to section →
Service layer pattern	Go to section →
Terraform (AWS infra)	Go to section →
Testing (Vitest)	Go to section →

Term	Jump to
Troubleshooting 25 fixes	Go to section →
TypeScript	Go to section →
Validation (Zod)	Go to section →
Zustand (state management)	Go to section →

Colophon

This book was produced from structured Markdown source with automated diagram rendering (Mermaid), professional print CSS, and PDF generation for consistent team distribution.

Typeface: System UI stack optimized for screen and print.

Diagrams: All figures rendered at publication resolution.

Source: the-complete-handbook.md

Complete Edition · Version 2.0 · 2026

— End of book —